COMMONWEALTH OF AUSTRALIA
Copyright Regulations 1969
This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice

Prepared by: [Arun Konagurthu]
Extended by: Taylor Kearney

FIT3155: Advanced Algorithms and Data Structures

Week 4: **Linear-time suffix tree construction**

Faculty of Information Technology, Monash University

What is covered in this lecture series?

Linear-time suffix tree construction

- Ukkonen algorithm (suffix tree construction)

Source material and recommended reading

- Dan Gusfield, Algorithms on Strings, Trees and Sequences, Cambridge University Press. (Chapter 5)
- Francisco Gomez Martin, Exact String Pattern Recognition
- Ukkonen, E. (1995). "On-line construction of suffix trees". Algorithmica 14 (3): 249–260.

Introduction – Jogging the memory

The pattern matching problem

Given a reference text **`txt[1...n]`**, preprocess **`txt`** such that any given pattern **`pat[1...m]`** can be searched in time proportional to the length of the pattern, $O(m)$.*

*Compare this with what we learnt from Z-algorithm and Boyer-Moore algorithm.

- Suffix trees (and suffix arrays) permit solving the above (and many other related) problems. They are very versatile.
- Suffix trees unravel the composition of strings, and permit efficient access to them.

String definitions – Jogging the memory

Given $\texttt{str}[1...n]$

- A **prefix** of $\texttt{str}[1..n]$ is a **substring** $\texttt{str}[1..i]$, for all $1 \leq i \leq n$.
- A **suffix** of $\texttt{str}[1..n]$ is a **substring** $\texttt{str}[j..n]$, for all $1 \leq j \leq n$.
- A **substring** of $\texttt{str}[1..n]$ is any $\texttt{str}[j..i]$, for all $1 \leq (j \leq i) \leq n$.
- Therefore,
  - In other words, a substring is a **prefix of a suffix**
  - (or, equivalently, a substring is a **suffix of a prefix**)

Suffixes of a string, example – Jogging the memory

Consider the suffixes of **str**$[1..n]$**$** $=$ **abaaba$**

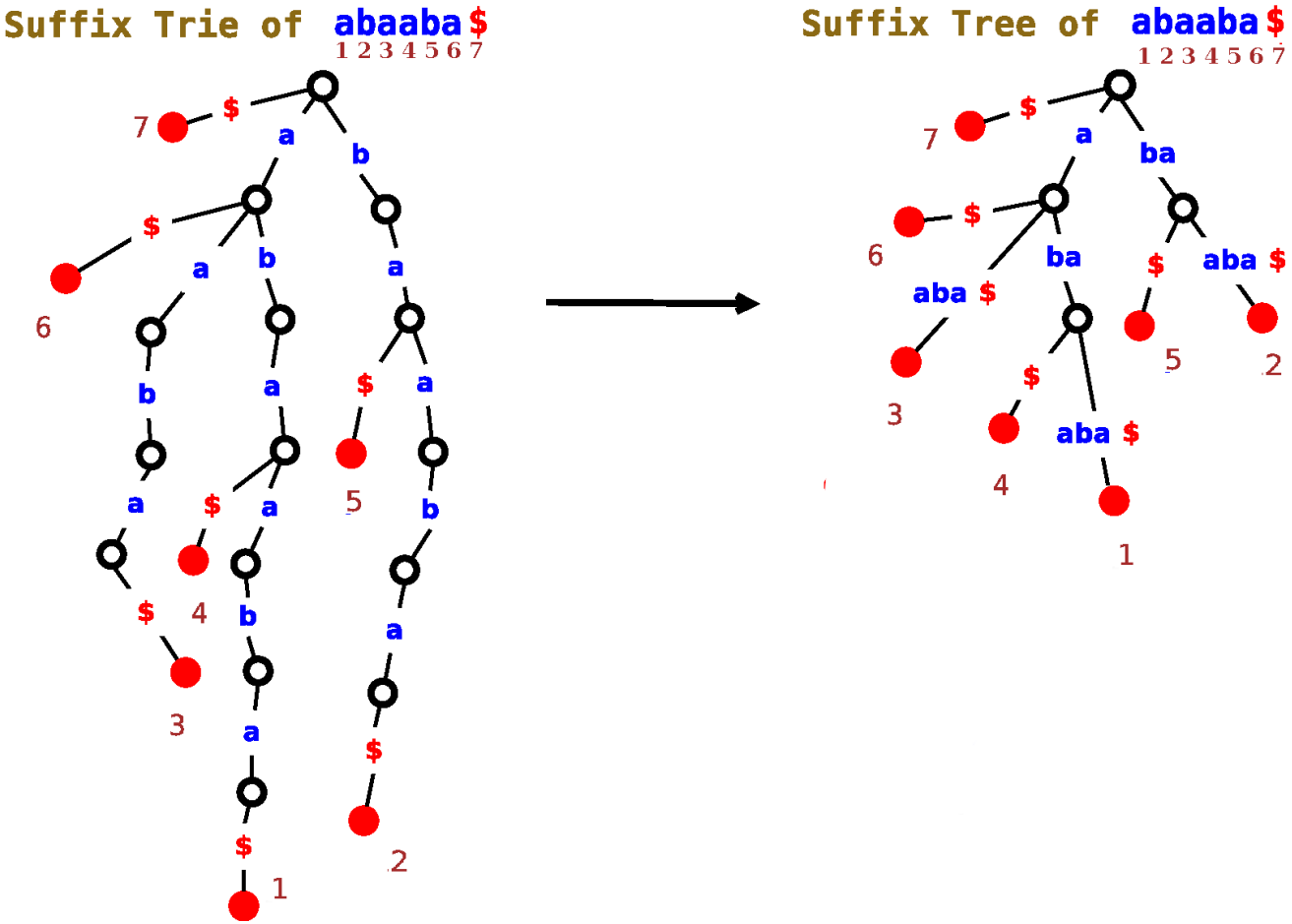
```
SUFFIX 1 = abaaba$
SUFFIX 2 = baaba$
SUFFIX 3 = aaba$
SUFFIX 4 = aba$
SUFFIX 5 = ba$
SUFFIX 6 = a$
SUFFIX 7 = $
```

The '$' symbol

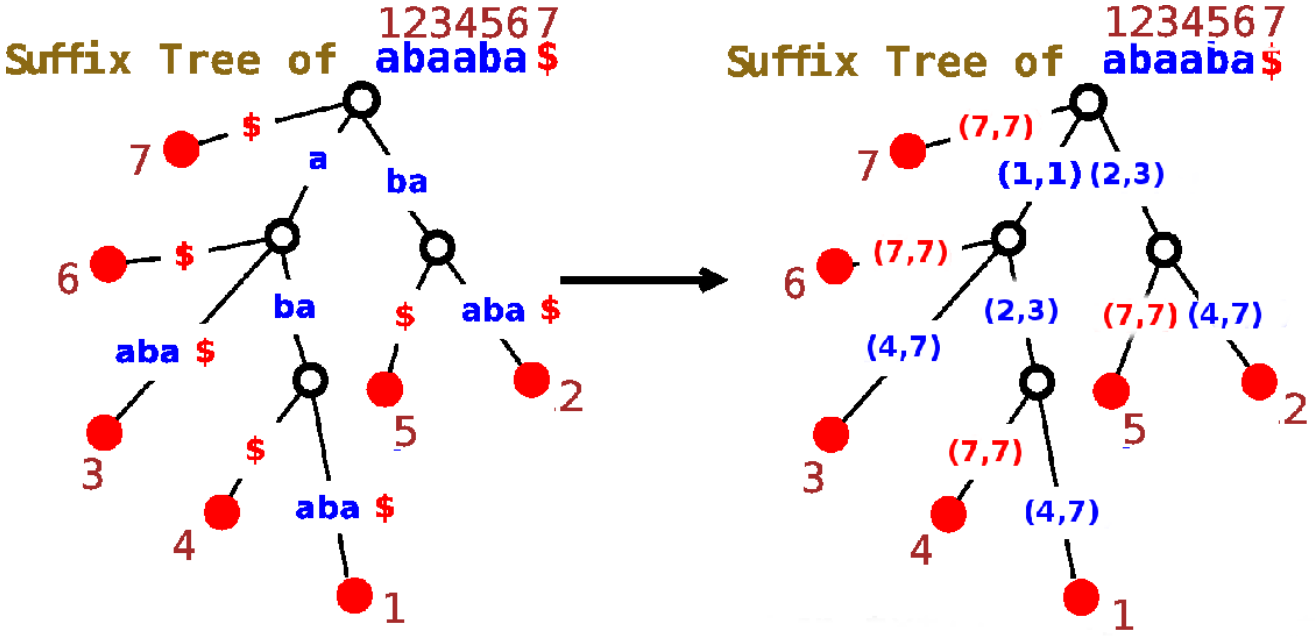
Note that **$** is a **special character** to denote the **end of the string**. It is often chosen to be a character that is **lexicographically smaller** than **all** other characters in the text.

Recall from FIT2004: Path compressed suffix tries = suffix trees – Jogging the memory

Suffix Trie (left) when path compressed is a Suffix Tree (right). All internal nodes with just one child get merged in path compression.



Efficient representation of suffix trees requires $O(n)$ space – (Self-study)



Storing the edge labels as substrings (see edge labels on the left) will result in worst-case $O(n^2)$ space. Since you have the original string stored, it suffices to store the substring information more efficiently in $O(n)$-spac using (start point, end point) indexes where <u>any one instance</u> of the substring appears (see edge labels on the right). Reason why?

Building a suffix tree by iterative insertion of suffixes

Consider suffixes of str= a b c a b $

```
SUFFIX 1: str[1..6] = a b c a b $
SUFFIX 2: str[2..6] = b c a b $
SUFFIX 3: str[3..6] = c a b $
SUFFIX 4: str[4..6] = a b $
SUFFIX 5: str[5..6] = b $
SUFFIX 6: str[6..6] = $
```

The substring labels in the figure above should be read in the direction of traversal from root (r) to the respective leaves.

Building a suffix tree by iterative insertion of suffixes

Consider suffixes of str= a b c a b $ (1 2 3 4 5 6)

```
SUFFIX 1: str[1..6] = a b c a b $
SUFFIX 2: str[2..6] = b c a b $
SUFFIX 3: str[3..6] = c a b $
SUFFIX 4: str[4..6] = a b $
SUFFIX 5: str[5..6] = b $
SUFFIX 6: str[6..6] = $
```

- Start with the (empty) root node of the suffix tree: (r)

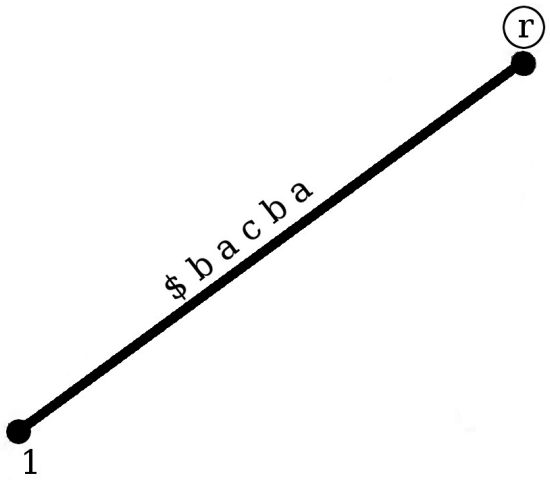
This insertion method requires $O(n^2)$ effort.

The substring labels in the figure above should be read in the direction of traversal from root (r) to the respective leaves.

Building a suffix tree by iterative insertion of suffixes

Consider suffixes of **str**= a b c a b $

```
SUFFIX 1: str[1..6] = a b c a b $
SUFFIX 2: str[2..6] = b c a b $
SUFFIX 3: str[3..6] = c a b $
SUFFIX 4: str[4..6] = a b $
SUFFIX 5: str[5..6] = b $
SUFFIX 6: str[6..6] = $
```



- Insert suffix 1 (**str**[1...]) into an empty tree.
- Call the resultant tree T_1.

This insertion method requires $O(n^2)$ effort.

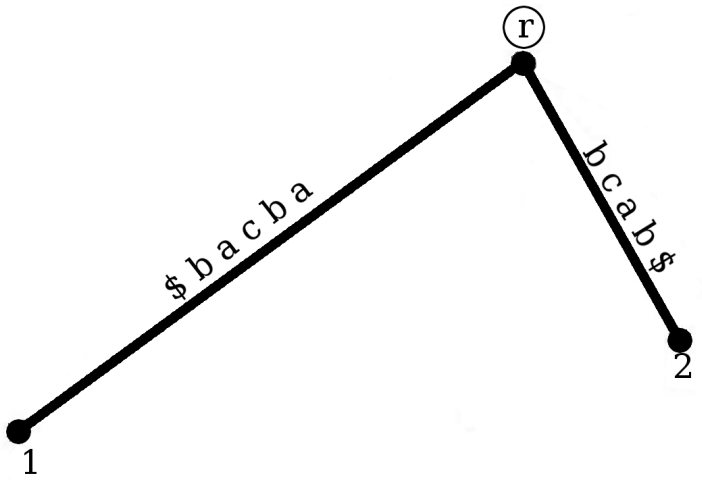
The substring labels in the figure above should be read in the direction of traversal from root (r) to the respective leaves.

Building a suffix tree by iterative insertion of suffixes

Consider suffixes of **str**= a b c a b $

1 2 3 4 5 6

```
SUFFIX 1: str[1..6] = a b c a b $
SUFFIX 2: str[2..6] = b c a b $
SUFFIX 3: str[3..6] = c a b $
SUFFIX 4: str[4..6] = a b $
SUFFIX 5: str[5..6] = b $
SUFFIX 6: str[6..6] = $
```



- Insert suffix 2 into T_1.
- Note **str**$[2....]$ is not a prefix of **str**$[1....]$.
- So create a new leaf node for **str**$[2..]$ suffix, branching off at r.
- Call resultant tree T_2

This insertion method requires $O(n^2)$ effort.

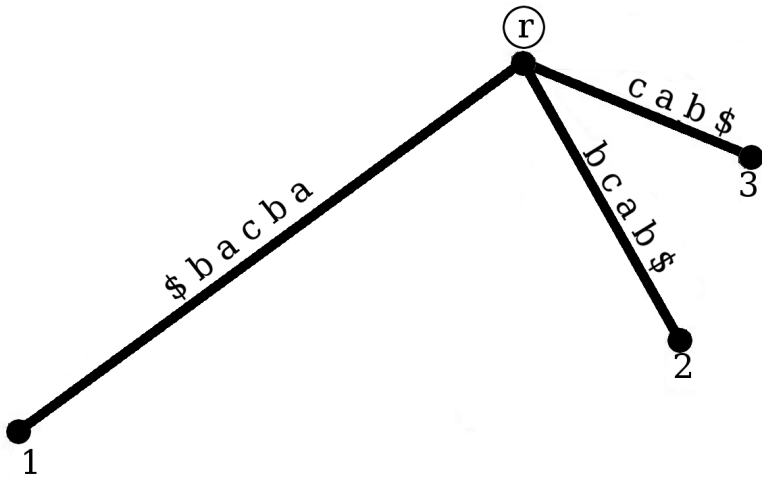
The substring labels in the figure above should be read in the direction of traversal from root r to the respective leaves.

Building a suffix tree by iterative insertion of suffixes

Consider suffixes of **str**= a b c a b $

1 2 3 4 5 6

```
SUFFIX 1: str[1..6] = a b c a b $
SUFFIX 2: str[2..6] = b c a b $
SUFFIX 3: str[3..6] = c a b $
SUFFIX 4: str[4..6] = a b $
SUFFIX 5: str[5..6] = b $
SUFFIX 6: str[6..6] = $
```



- Insert suffix 3 into T_2.
- Note **str**$[3....]$ is not a prefix of **str**$[1....]$ or **str**$[2....]$.
- So create a new leaf node for **str**$[3..]$ suffix, again branching off at r.
- Call resultant tree T_3

This insertion method requires $O(n^2)$ effort.

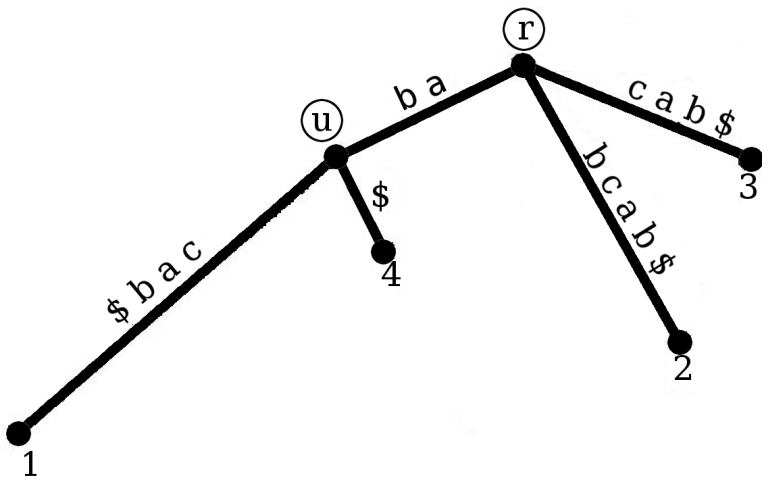
The substring labels in the figure above should be read in the direction of traversal from root r to the respective leaves.

Building a suffix tree by iterative insertion of suffixes

Consider suffixes of **str**= a b c a b $

1 2 3 4 5 6

```
SUFFIX 1: str[1..6] = a b c a b $
SUFFIX 2: str[2..6] = b c a b $
SUFFIX 3: str[3..6] = c a b $
SUFFIX 4: str[4..6] = a b $
SUFFIX 5: str[5..6] = b $
SUFFIX 6: str[6..6] = $
```



- Inserting suffix 4 into T_3.
- Note **str**$[4..5]$ is the longest common prefix shared with the suffix **str**$[1....]$.
- So, a new node (u) is **inserted** ...
- ... along the edge between (r) and the leaf node 1...
- ... with another edge branching off (u) to the new leaf node 4
- Call resultant tree T_4

This insertion method requires $O(n^2)$ effort.

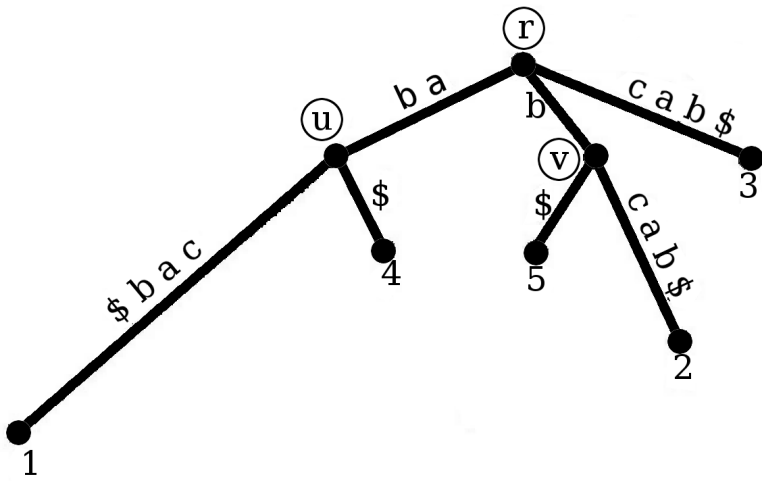
The substring labels in the figure above should be read in the direction of traversal from root (r) to the respective leaves.

Building a suffix tree by iterative insertion of suffixes

Consider suffixes of **str**= a b c a b $

1 2 3 4 5 6

```
SUFFIX 1: str[1..6] = a b c a b $
SUFFIX 2: str[2..6] = b c a b $
SUFFIX 3: str[3..6] = c a b $
SUFFIX 4: str[4..6] = a b $
SUFFIX 5: str[5..6] = b $
SUFFIX 6: str[6..6] = $
```



- Insert suffix 5 into T_4.
- Note **str**$[5..5]$ is the longest common prefix shared with suffix **str**$[2..]$.
- So, a new node (v) is **inserted**...
- ... along the edge between (r) and the leaf node 2...
- ... with another edge branching out from (v) to the new leaf node 5
- Call resultant tree T_5

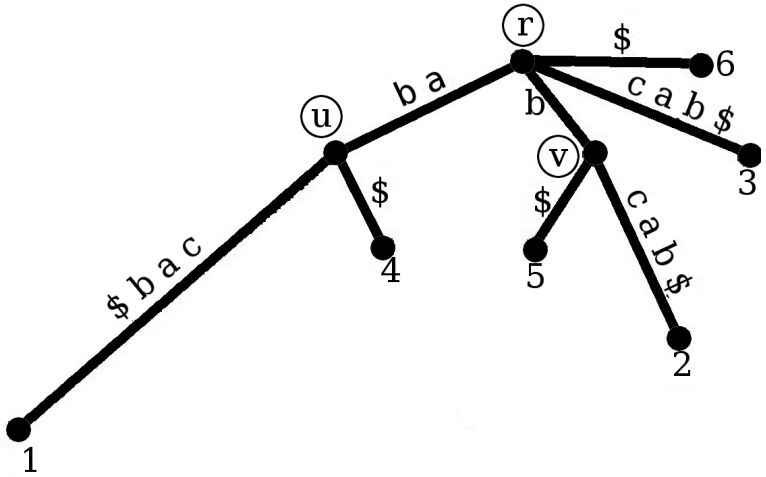
This insertion method requires $O(n^2)$ effort.

The substring labels in the figure above should be read in the direction of traversal from root (r) to the respective leaves.

Building a suffix tree by iterative insertion of suffixes

Consider suffixes of **str**= a b c a b $

```
SUFFIX 1: str[1..6] = a b c a b $
SUFFIX 2: str[2..6] = b c a b $
SUFFIX 3: str[3..6] = c a b $
SUFFIX 4: str[4..6] = a b $
SUFFIX 5: str[5..6] = b $
SUFFIX 6: str[6..6] = $
```



- Insert suffix 6 into T_5.
- Note the suffix **str**$[6..6]$ denotes the special terminal character $.
- This creates a new isolated edge branching off the root (r).

This insertion method requires $O(n^2)$ effort.

The substring labels in the figure above should be read in the direction of traversal from root (r) to the respective leaves.

Efficient suffix tree construction using **Ukkonen's** algorithm

Linear Time Suffix Tree construction

Esko Ukkonen in 1995 gave a very clever algorithm to construct suffix trees in linear run time.

Ukkonen, E. (1995). "On-line construction of suffix trees". Algorithmica 14 (3): 249–260.

Google

Esko Ukkonen

Finnish computer scientist

Esko Juhani Ukkonen is a Finnish theoretical computer scientist known for his contributions to string algorithms, and particularly for Ukkonen's algorithm for suffix tree construction. He is a professor at the University of Helsinki. Wikipedia

Born: 26 January 1950 (age 69 years), Savonlinna, Finland

Alma mater: University of Helsinki

Known for: Ukkonen's algorithm

Doctoral advisor: Martti Tienari

Ukkonen's linear-time algorithm – introduction

Ukkonen's algorithm exploits the properties inherent in any suffix tree, and benefits from several 'tricks' to achieve a linear run time.

Ukkonen's algorithms uses the following main ideas:

1. Iteratively construct '**`implicit suffix tree`**' corresponding to the prefixes of the actual string.
   - the actual/explicit suffix tree is computed in the last iteration (phase).
2. enhance this **`implicit suffix tree`** using '**`suffix links`**'.
   - this helps make the traversals on the implicit tree much faster.
3. gain from a set of implementational 'tricks':
   - these tricks avoid unnecessary computations, thus speeding up the algorithm drastically.

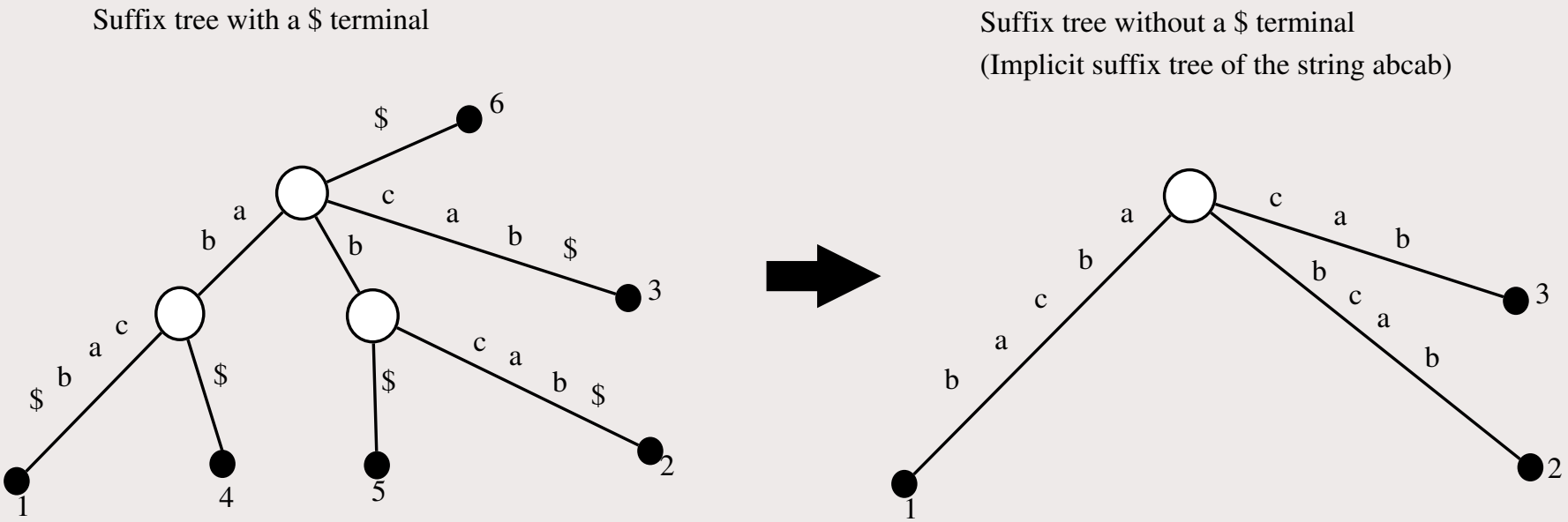
Only when all these ideas/elements are used together, Ukkonen's algorithm achieves a linear-time construction of suffix trees.

What is an **implicit** suffix tree?

Implicit suffix tree is any suffix tree constructed without the terminal (**$**) character.

Example

Regular suffix tree of **str**=a b c a b $ (1 2 3 4 5 6) and its implicit suffix tree without the terminal **$** character.



Ukkonen's algorithm builds implicit suffix trees incrementally in **phases**

Given a string **str**$[1..n]$:

- Ukkonen's algorithm proceeds over n "**phases**"
  - End of each **phase** i, you have **implicitST**$_i$ (i.e., the **implicit suffix tree** containing all suffixes of the **prefix** **str**$[1..i]$)
  - **implicitST**$_{i+1}$ is built upon the information already in **implicitST**$_i$, using what is called as **suffix extensions** (see next slide).

Each phase involves suffix extensions

Suffix extension steps in each phase

- In any phase $i+1$, the suffixes in **implicitST**$_i$ (from previous phase i) undergo **suffix extensions** to accommodate the **new character**, **str**$[i+1]$.
- Thus, extending any suffix starting at position j in the current phase $i+1$ involves:
  - finding the end of the path from root node (r) corresponding to the suffix **str**$[j \ldots i]$ in the current state of the implicit suffix tree, and
  - extending the end of **str**$[j \ldots i]$ path by appending **str**$[i+1]$ to that (growing) suffix.

Algorithm at an very high level

Initialize a root node. Call this $\texttt{implicitST}_0$.

For i from 0 to $n-1$

Begin **PHASE** $i+1$

- For j from 1 to $i+1$

  Begin **SUFFIX EXTENSION** j

  - ★ Find end of path from root denoting $\texttt{str}[j..i]$ in the current state of the suffix tree.
  - ★ extend the end of $\texttt{str}[j \ldots i]$ path by appending $\texttt{str}[i+1]$ to that (growing) suffix startng at j.

  End of extension step j (i.e., $\texttt{str}[j..i+1]$ extension computed).

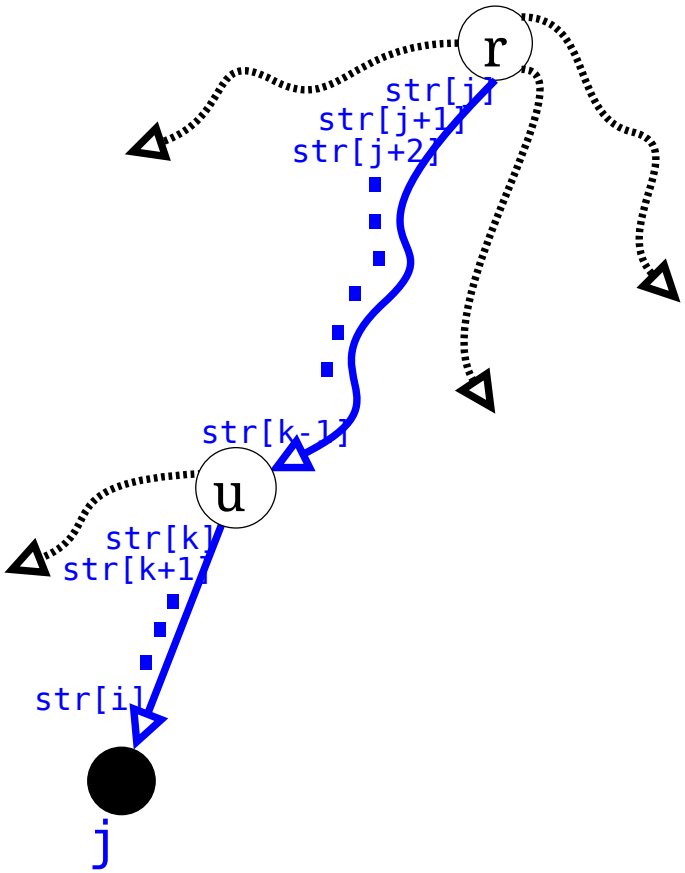
End of phase $i+1$ ($\texttt{implicitST}_{i+1}$ computed)

Suffix extension rules – Rule 1 of 3

RULE 1 of 3

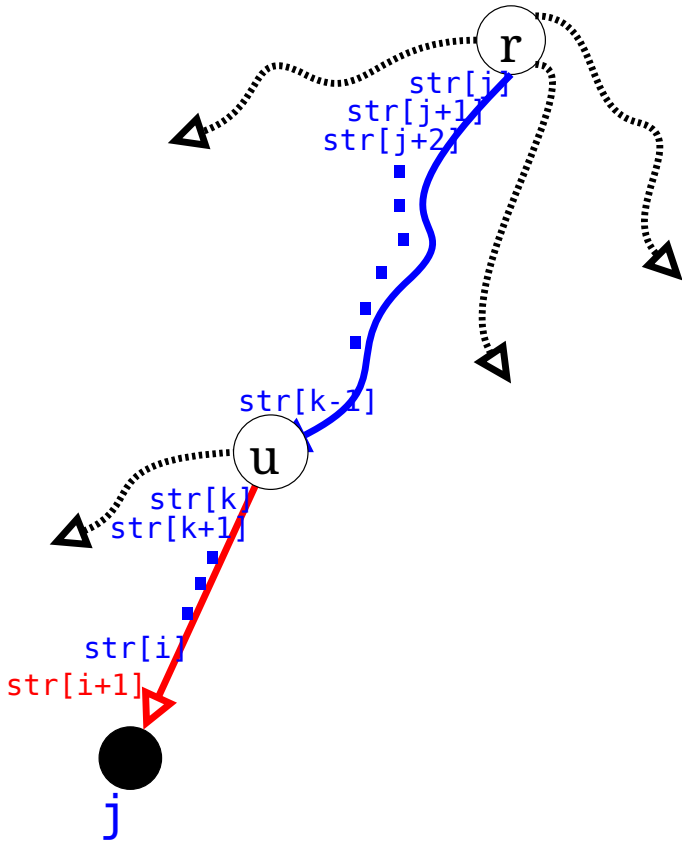
If the path $\texttt{str}[j..i]$ in $\texttt{implicitST}_i$ **ends at a leaf**, adjust the **label of the edge** to that leaf to account for the added character $\texttt{str}[i+1]$.

Before suffix extension



$\longrightarrow$

After suffix extension

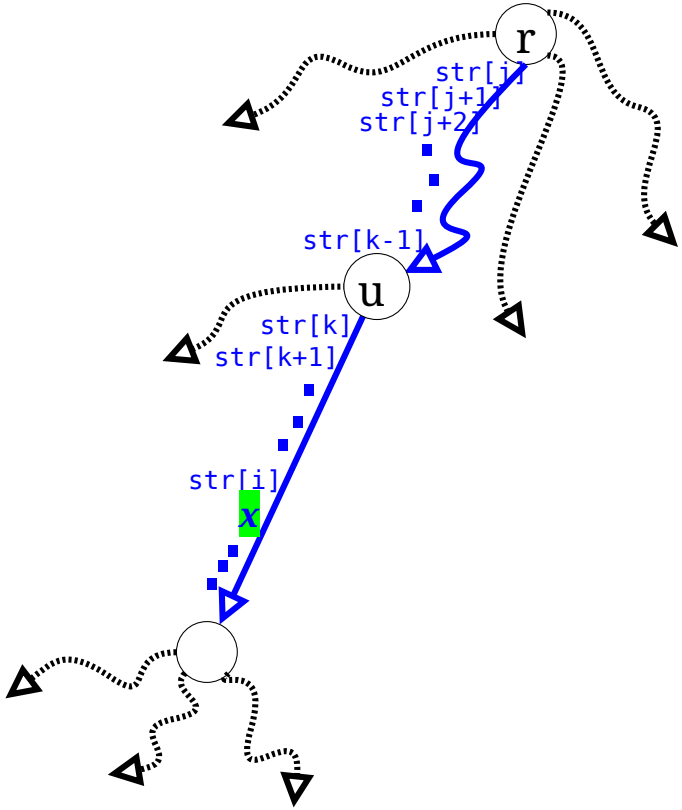


Suffix extension rules – Rule 2 of 3

RULE 2 of 3 (regular scenario)

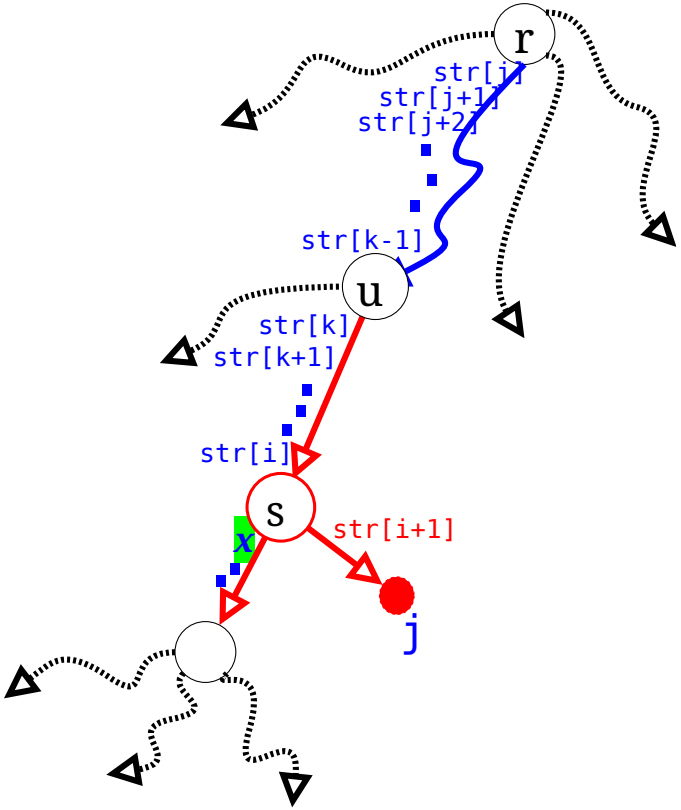
If the path $\texttt{str}[j..i]$ in $\texttt{implicitST}_i$ **does NOT** end at a leaf, but rather ends in between an edge below some internal node (u), and the next character on the same edge is some $x \neq \texttt{str}[i+1]$, then standing on (u), split the edge after $\texttt{str}[..i]$ and create a new internal node (s), and hang a new leaf labelled j off (s), with the single character substring $\texttt{str}[i+1]$ as the edge label.

Before suffix extension



$\longrightarrow$

After suffix extension

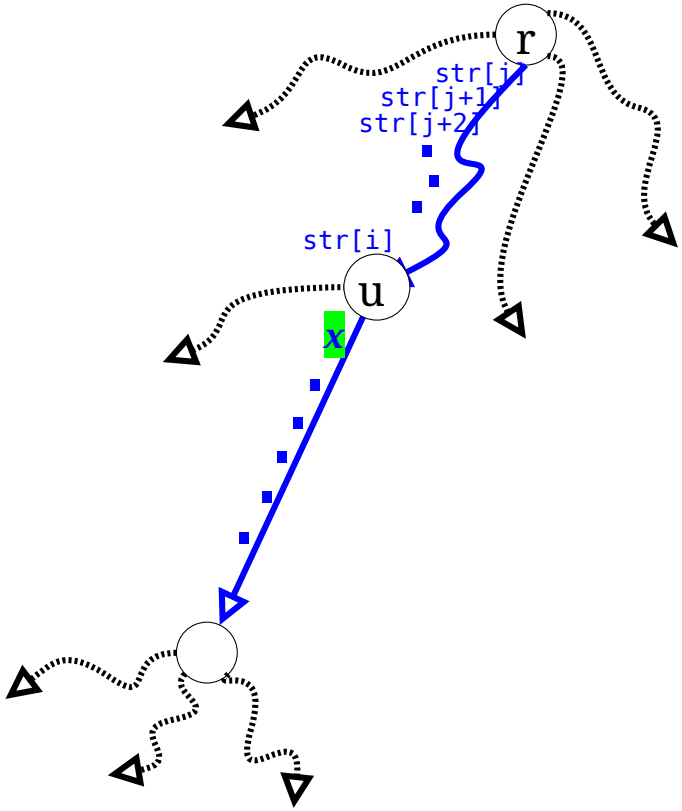


Suffix extension rules – Rule 2 of 3 (alternate scenario)

RULE 2 of 3 – an alternative scenario

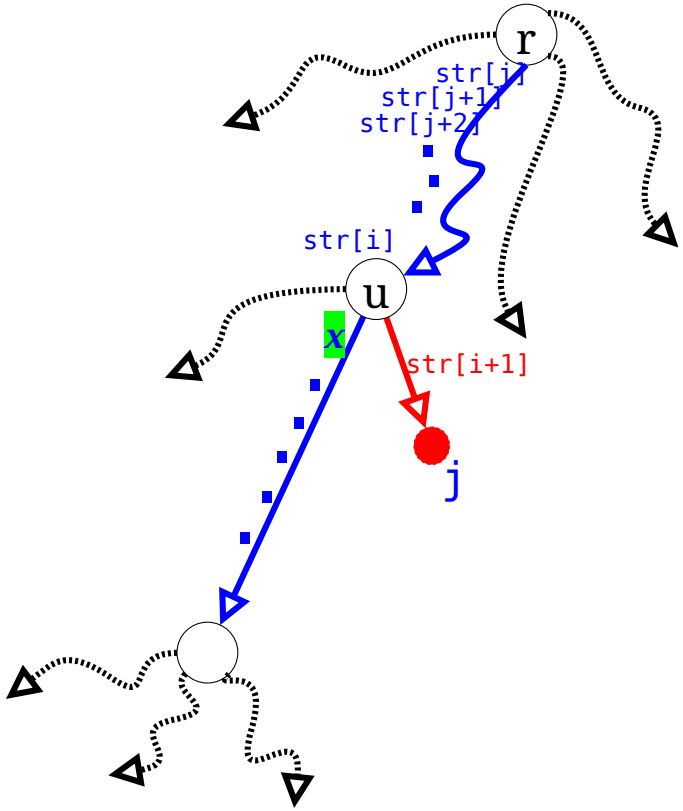
If the path $\texttt{str}[j..i]$ in $\texttt{implicitST}_i$ **does NOT** end at a leaf, and the next character in the existing path is some $x \neq \texttt{str}[i+1]$, and $\texttt{str}[i]$ and x are separated by some existing node (u), then hang a new leaf labelled j off (u), with the single character substring $\texttt{str}[i+1]$ as the edge label.

Before suffix extension



$\longrightarrow$

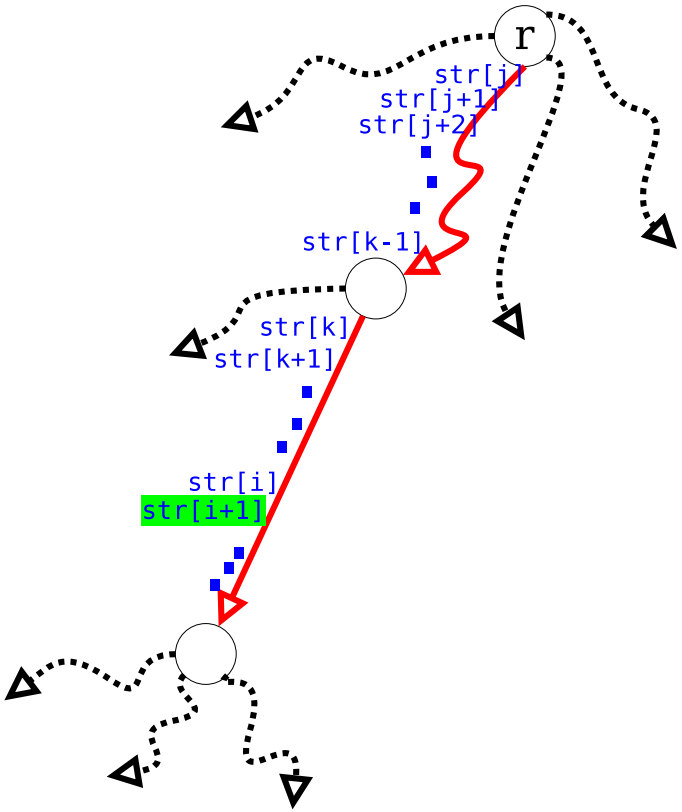
After suffix extension



Suffix extension rules – Rule 3 of 3

RULE 3 of 3

If the path **str**$[j..i]$ in **implicitST**$_i$ **does NOT** end at a leaf, but is within some edge label, and the next character in that path **is** **str**$[i+1]$, then **str**$[i+1]$ is already in the tree. **No further action needed.**

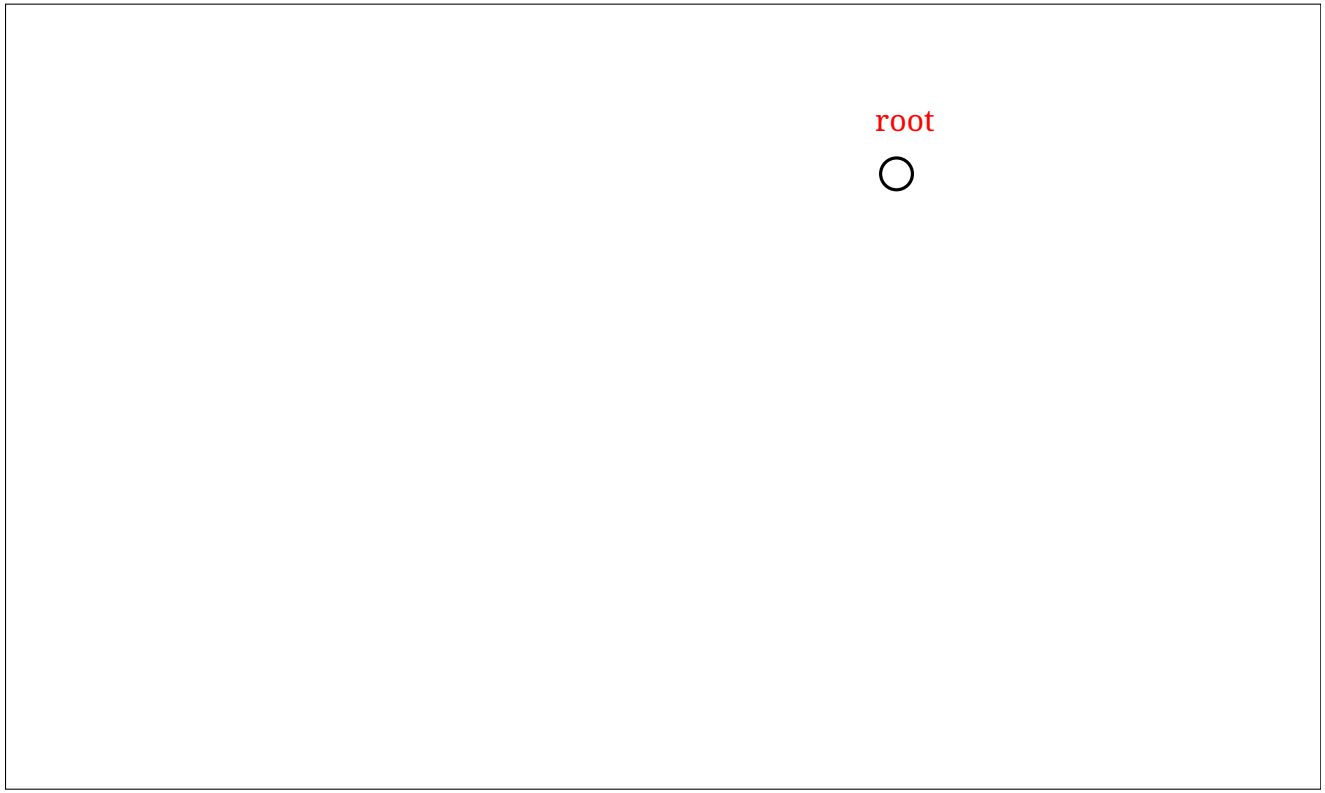


$\longrightarrow$ **No further action needed.**

Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

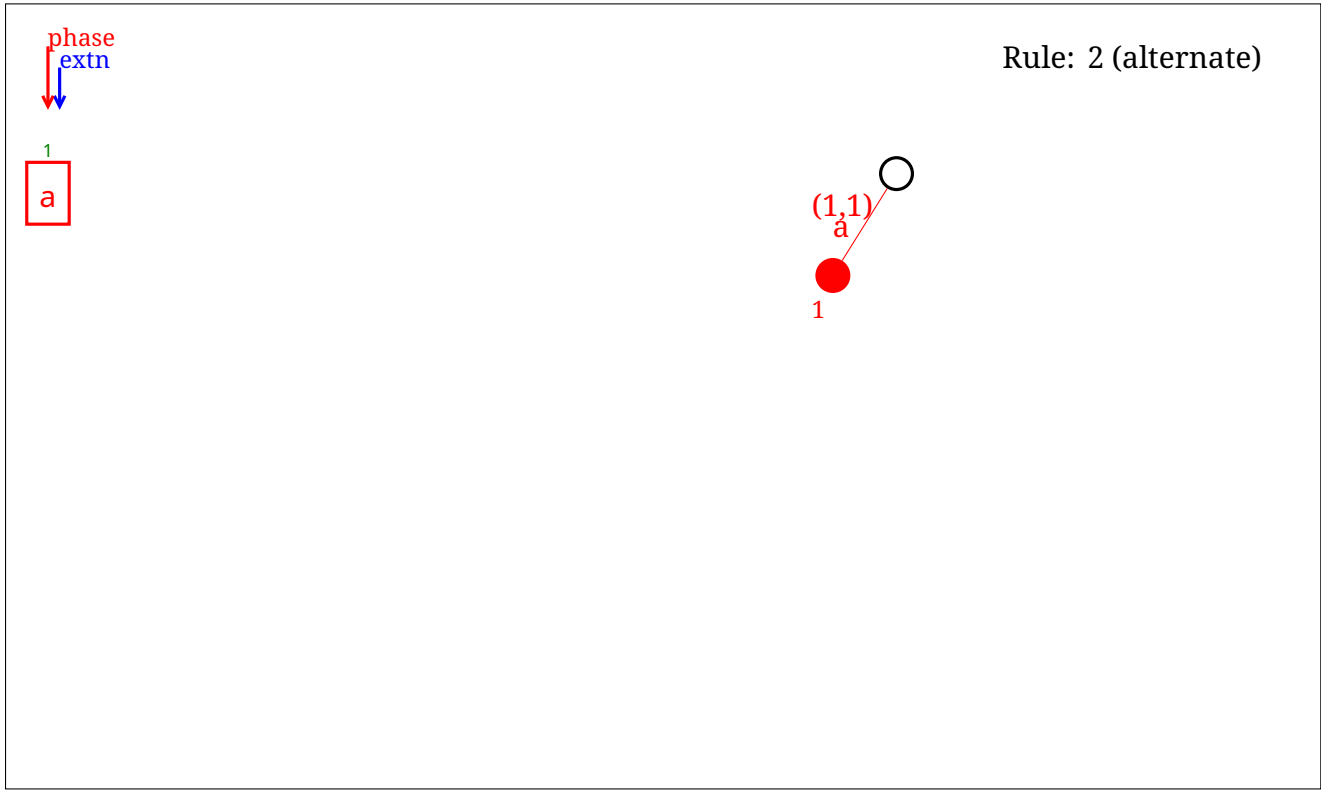
Initial state before the construction starts



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

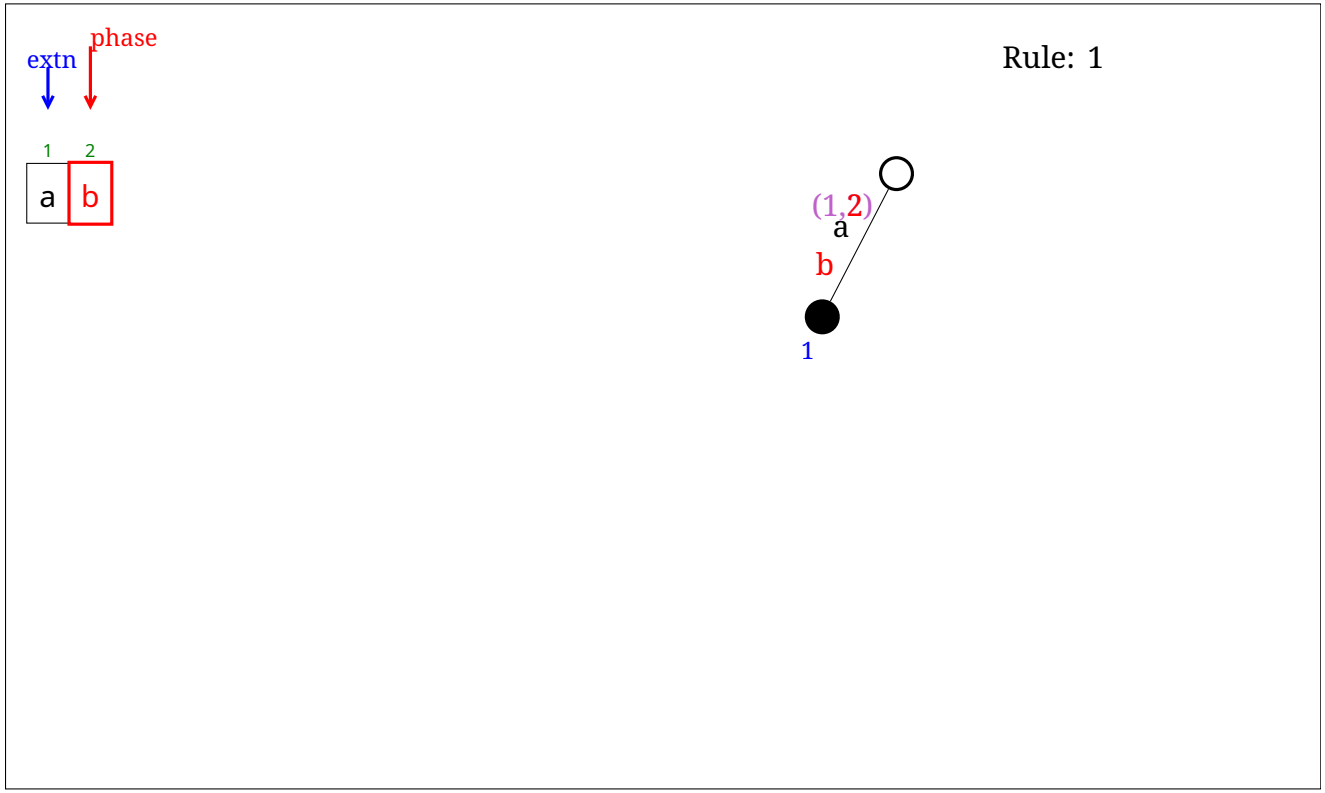
After Phase $i = 1$, Extension $j = 1$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

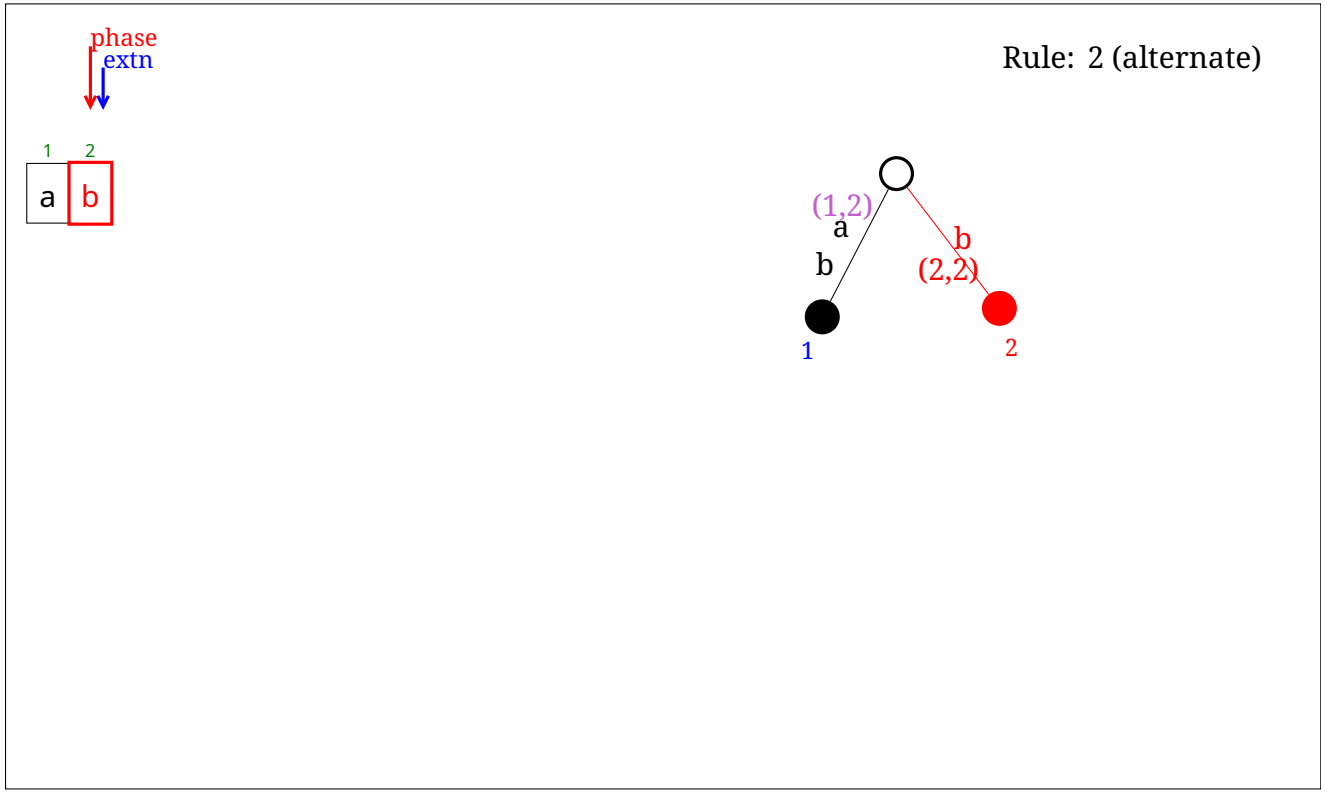
After Phase $i = 2$, Extension $j = 1$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

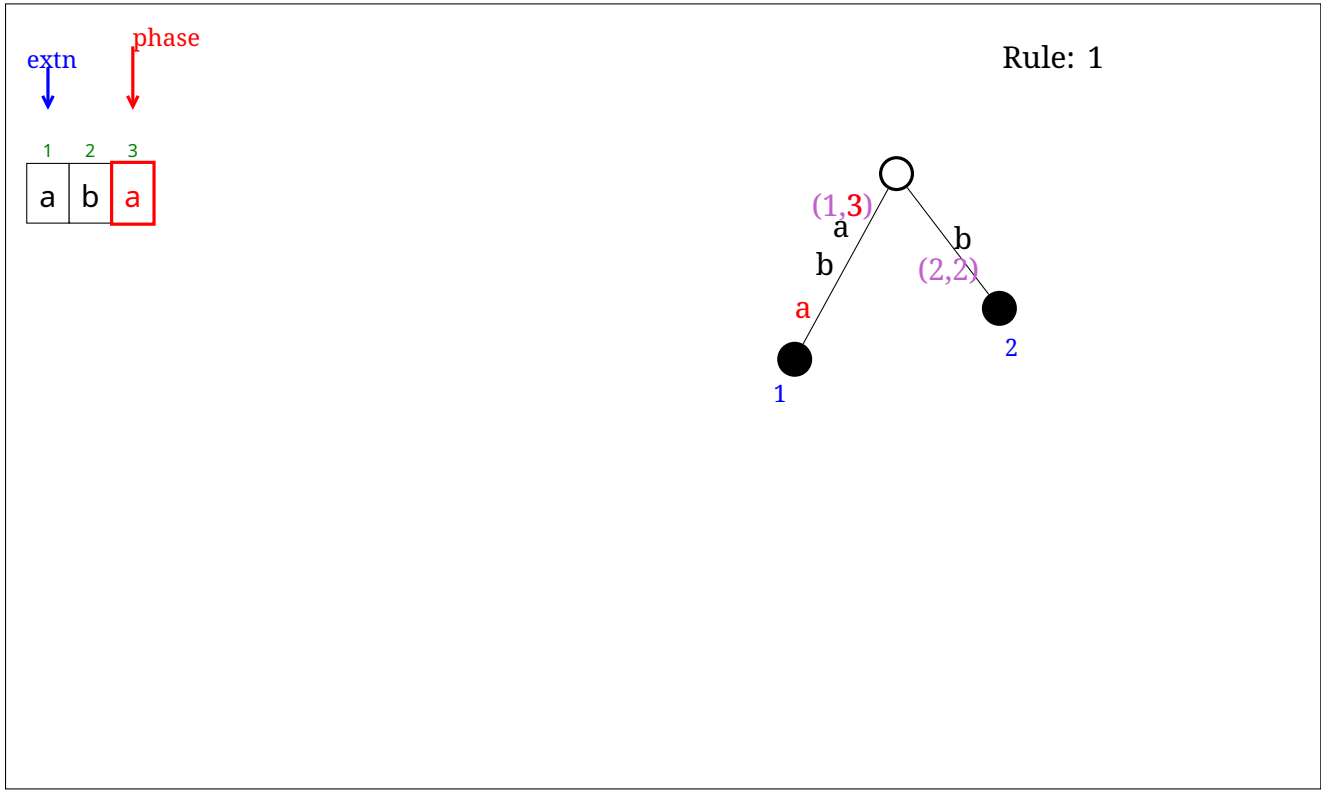
After Phase $i = 2$, Extension $j = 2$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

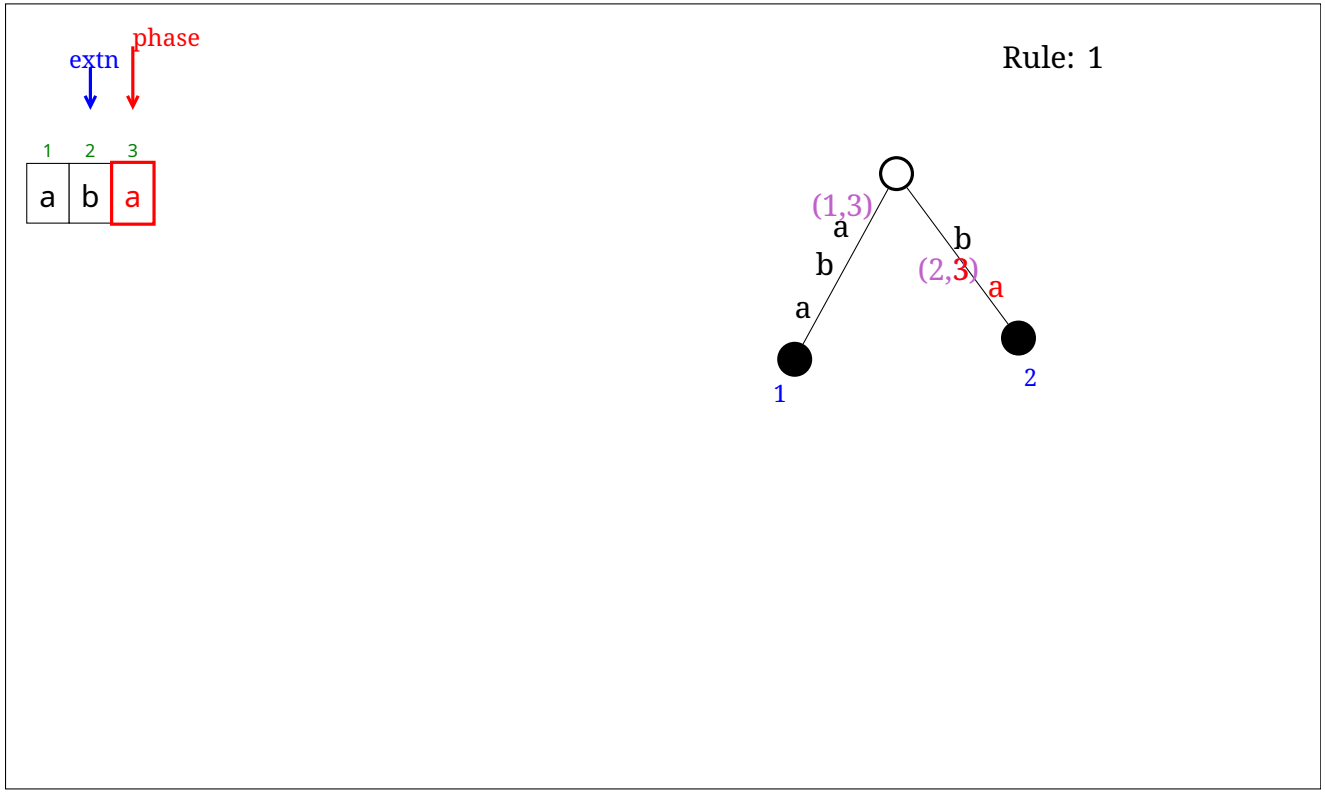
After Phase $i = 3$, Extension $j = 1$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

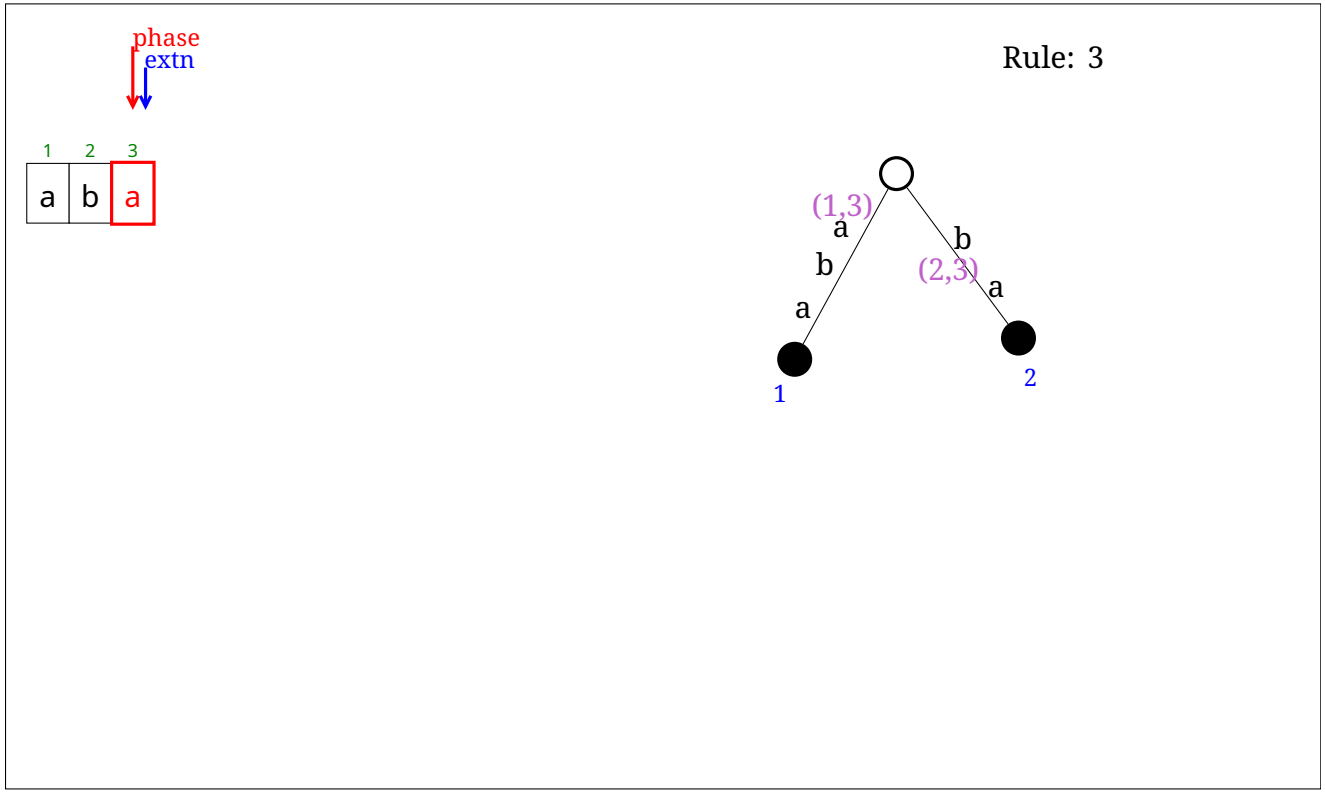
After Phase $i = 3$, Extension $j = 2$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\text{a}}\ \overset{2}{\text{b}}\ \overset{3}{\text{a}}\ \overset{4}{\text{a}}\ \overset{5}{\text{b}}\ \overset{6}{\text{a}}\ \overset{7}{\$}$$

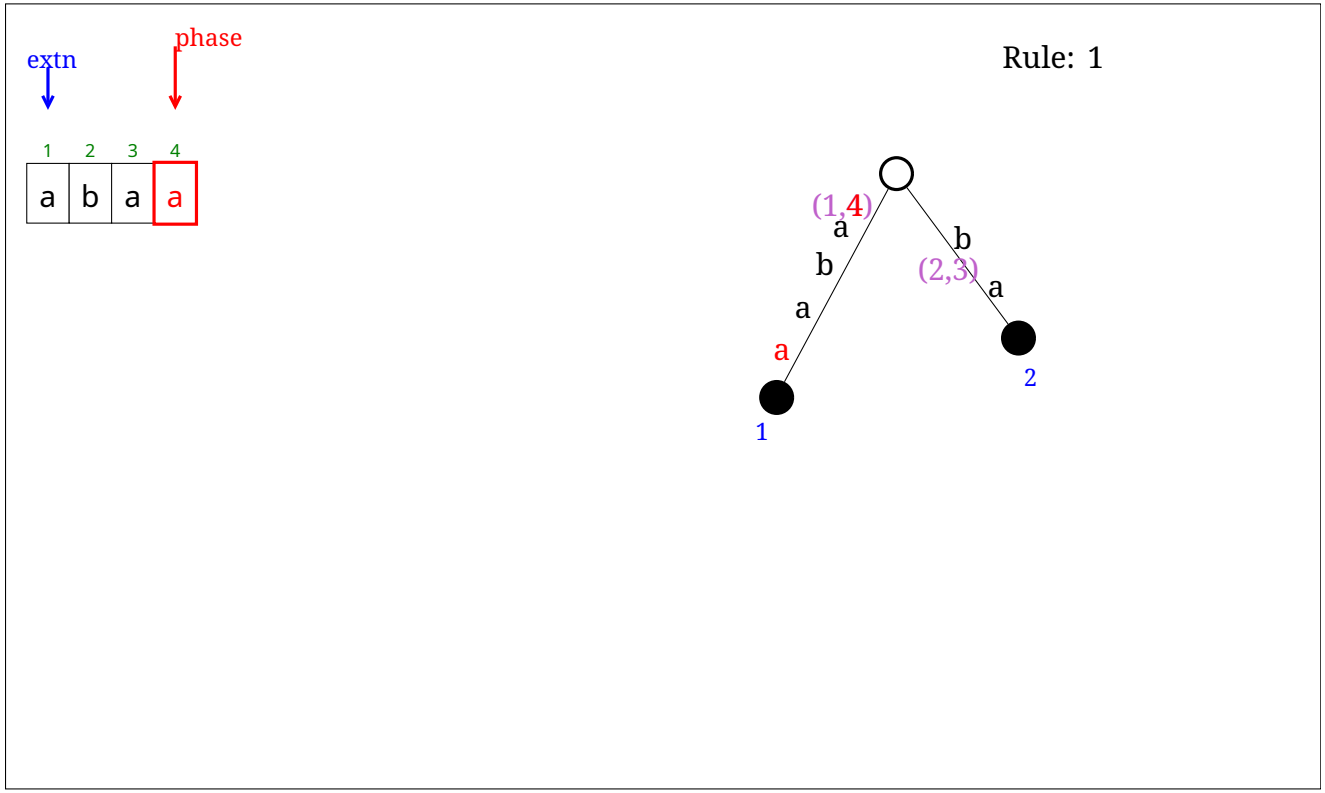
After Phase $i = 3$, Extension $j = 3$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

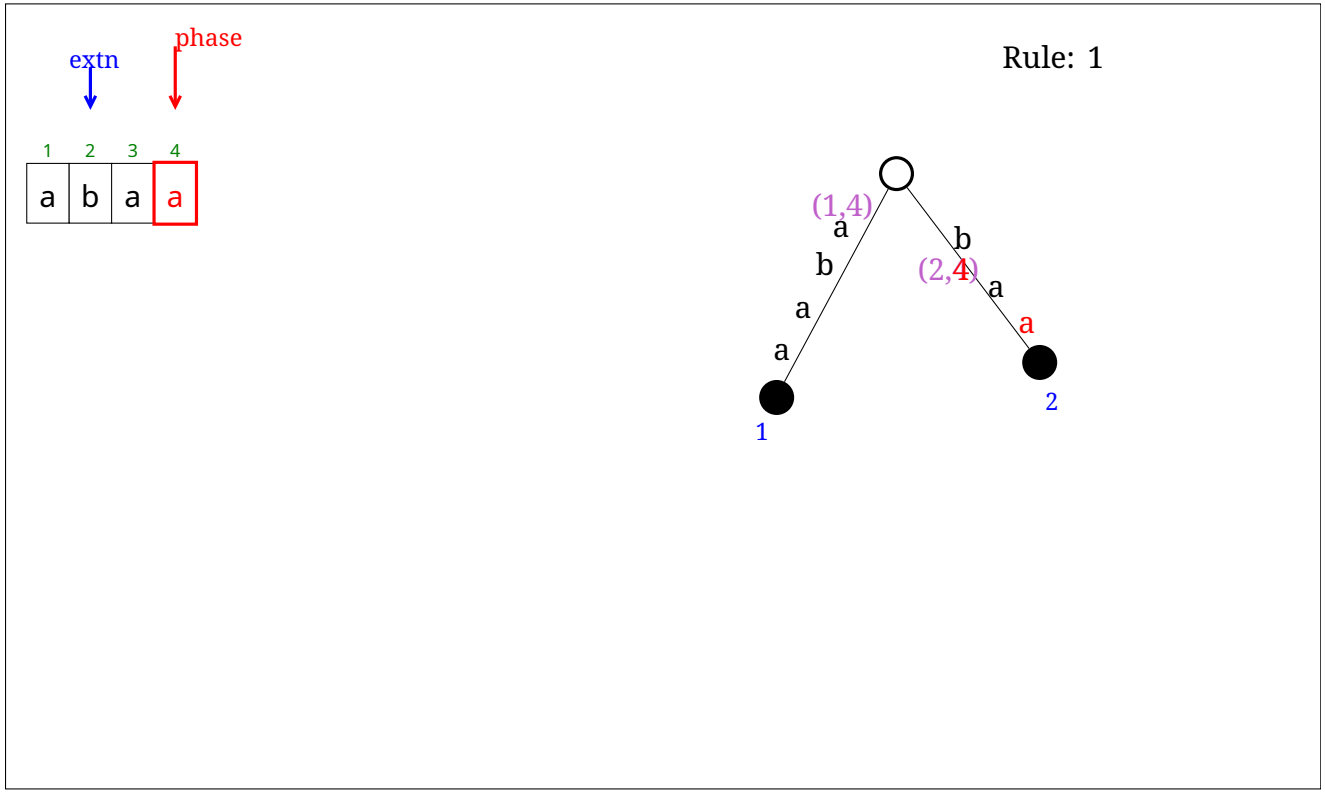
After Phase $i = 4$, Extension $j = 1$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

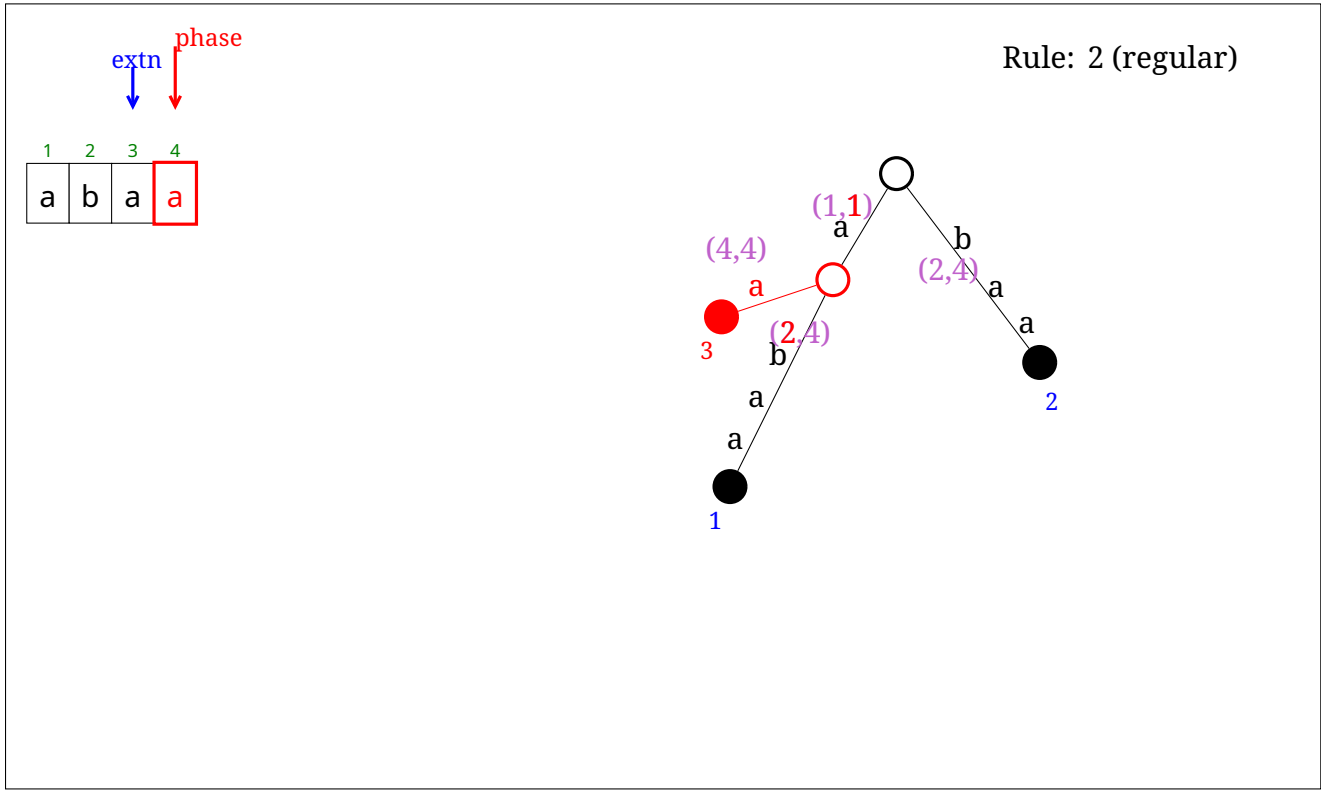
After Phase $i = 4$, Extension $j = 2$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

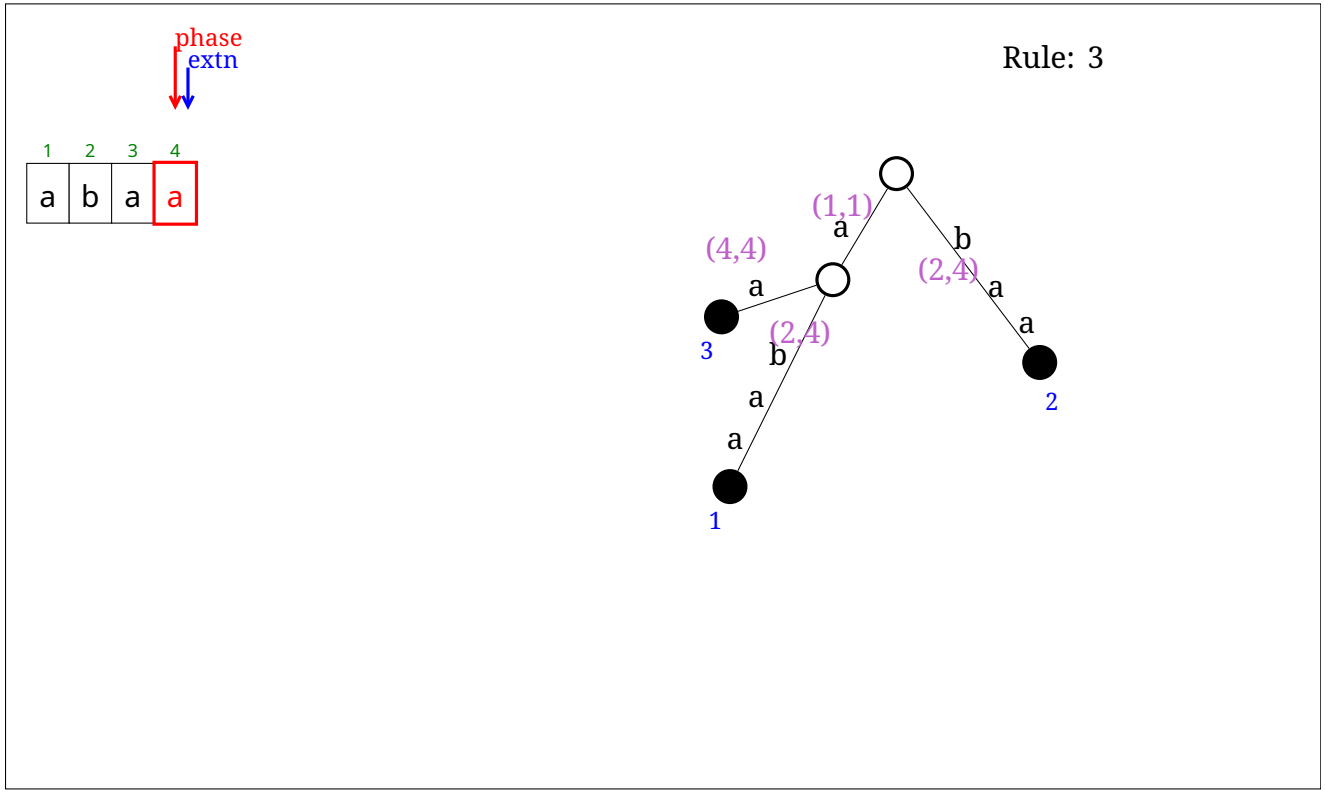
After Phase $i = 4$, Extension $j = 3$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

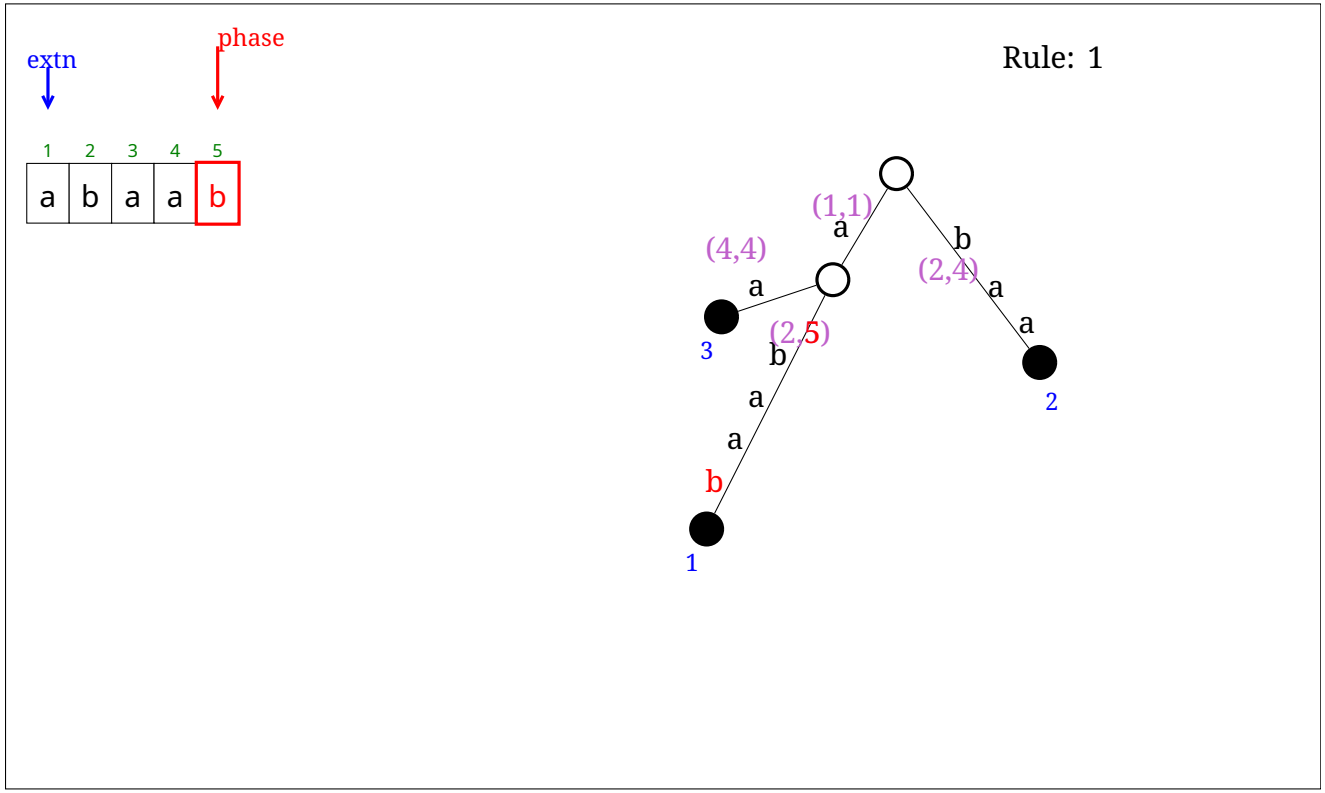
After Phase $i = 4$, Extension $j = 4$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

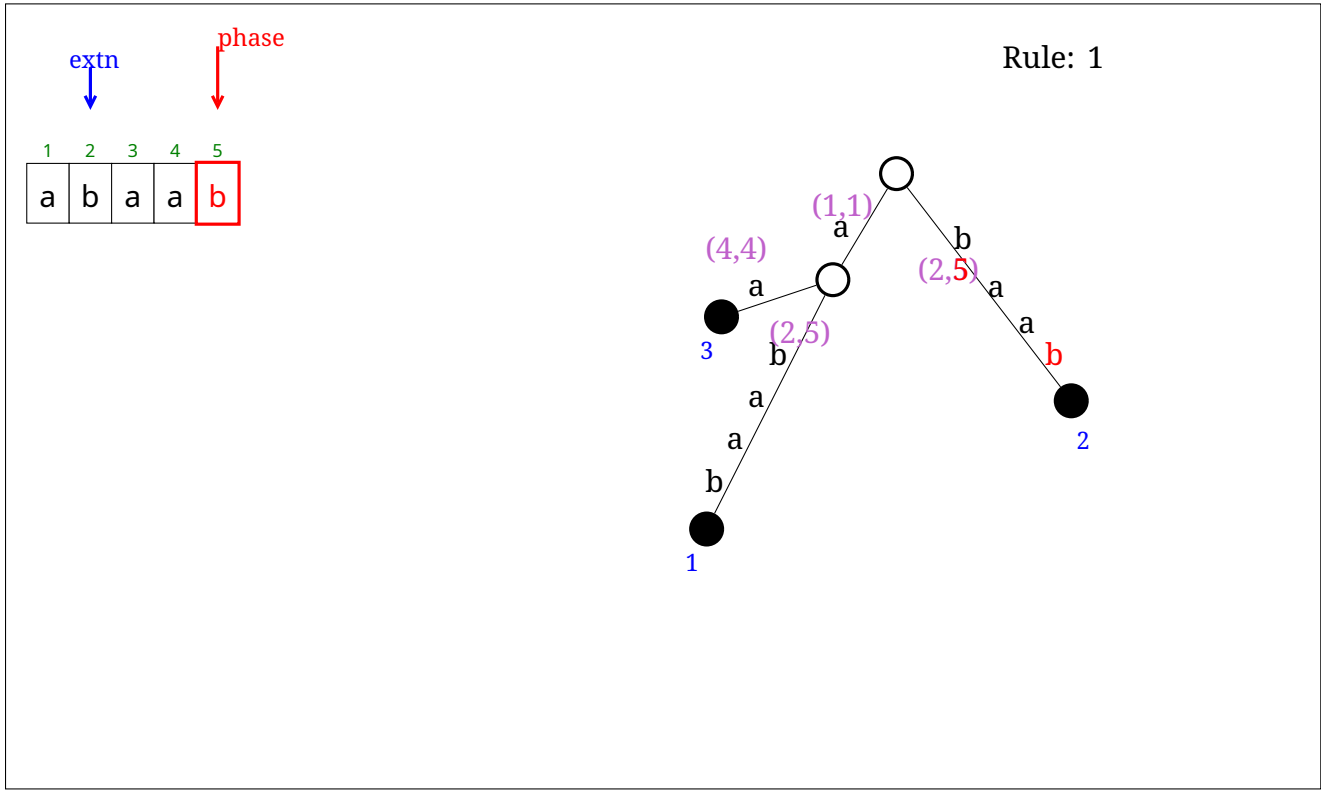
After Phase $i = 5$, Extension $j = 1$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

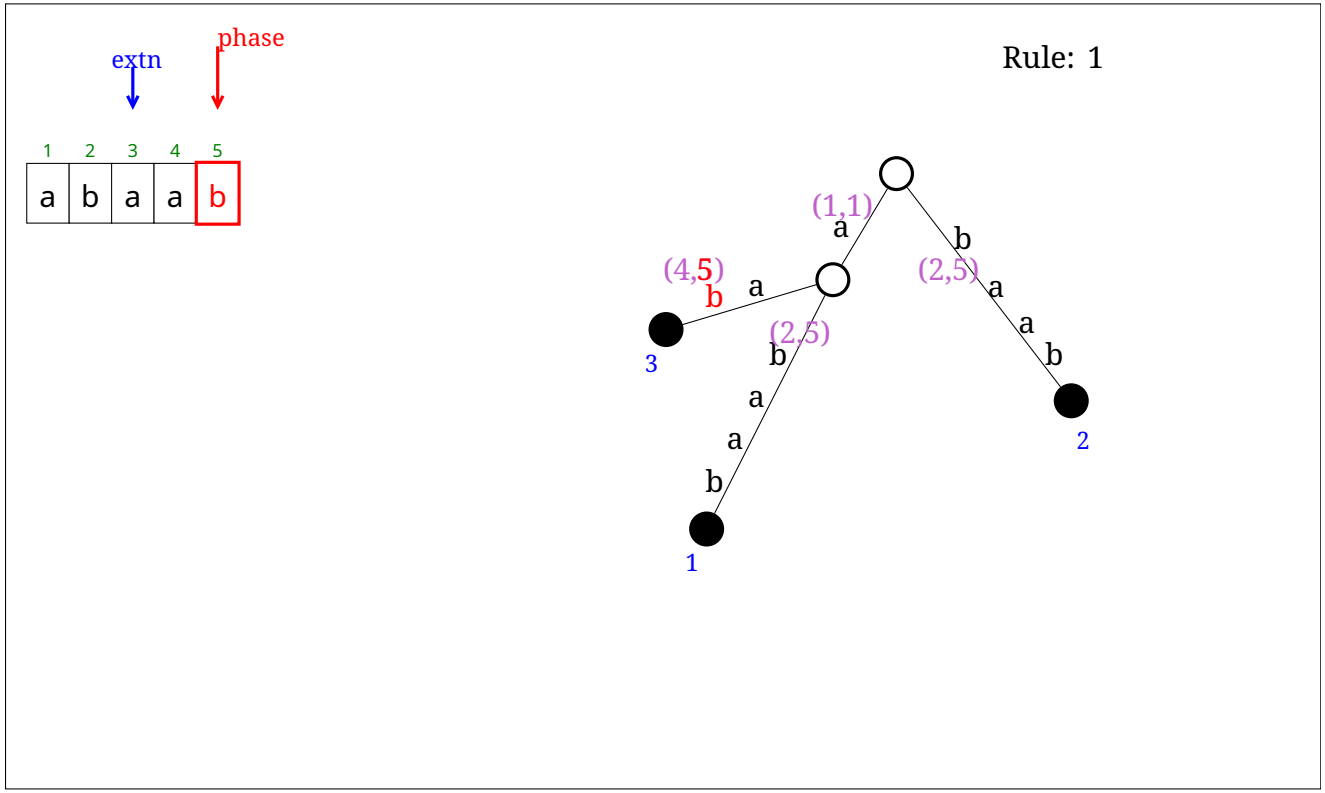
After Phase $i = 5$, Extension $j = 2$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

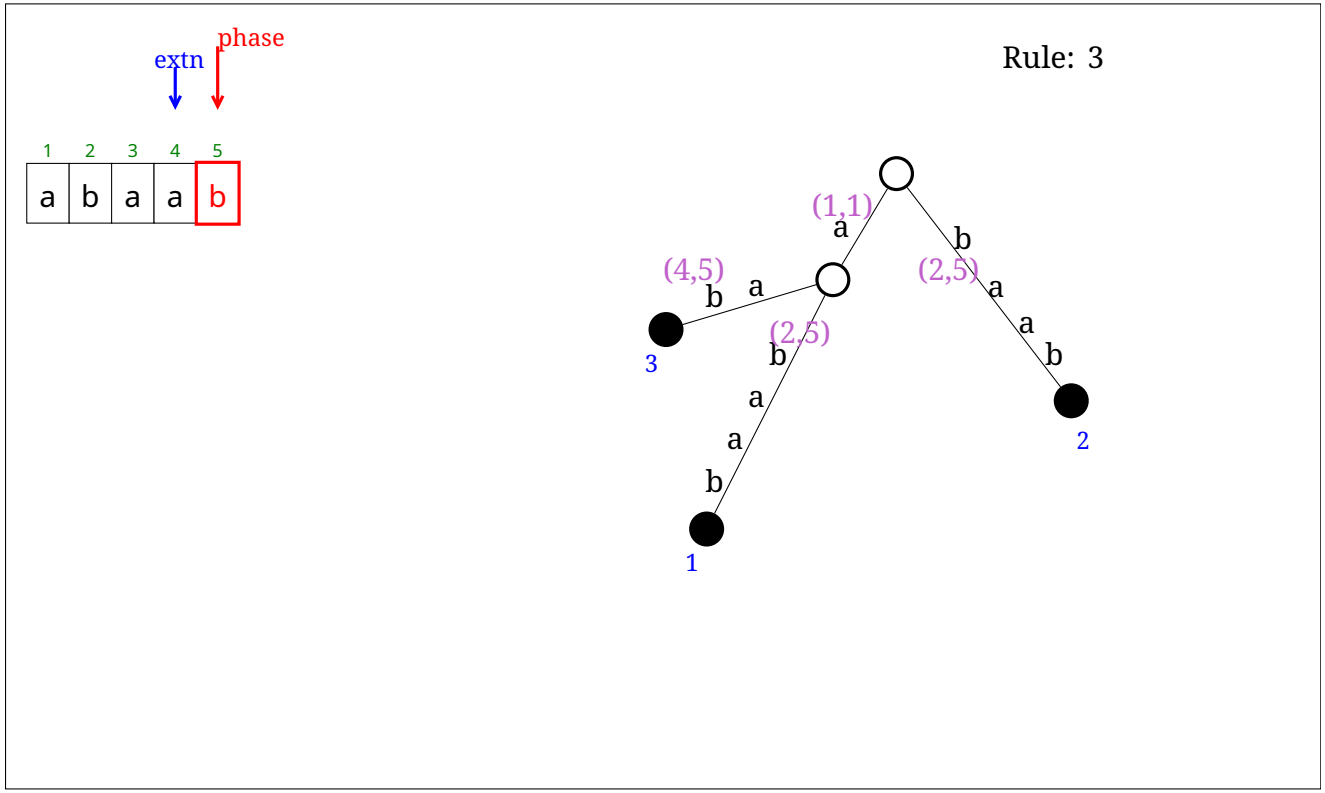
After Phase $i = 5$, Extension $j = 3$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\text{a}}\ \overset{2}{\text{b}}\ \overset{3}{\text{a}}\ \overset{4}{\text{a}}\ \overset{5}{\text{b}}\ \overset{6}{\text{a}}\ \overset{7}{\$}$$

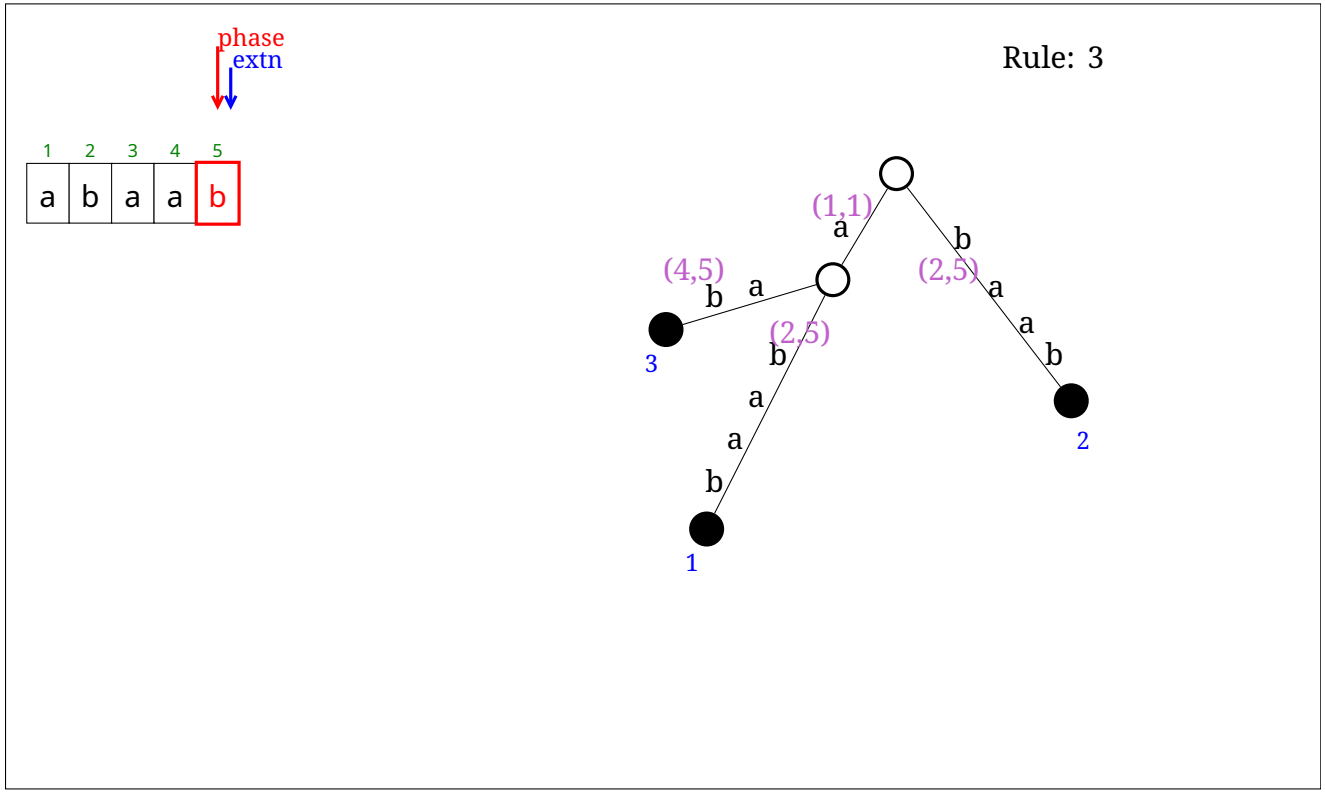
After Phase $i = 5$, Extension $j = 4$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

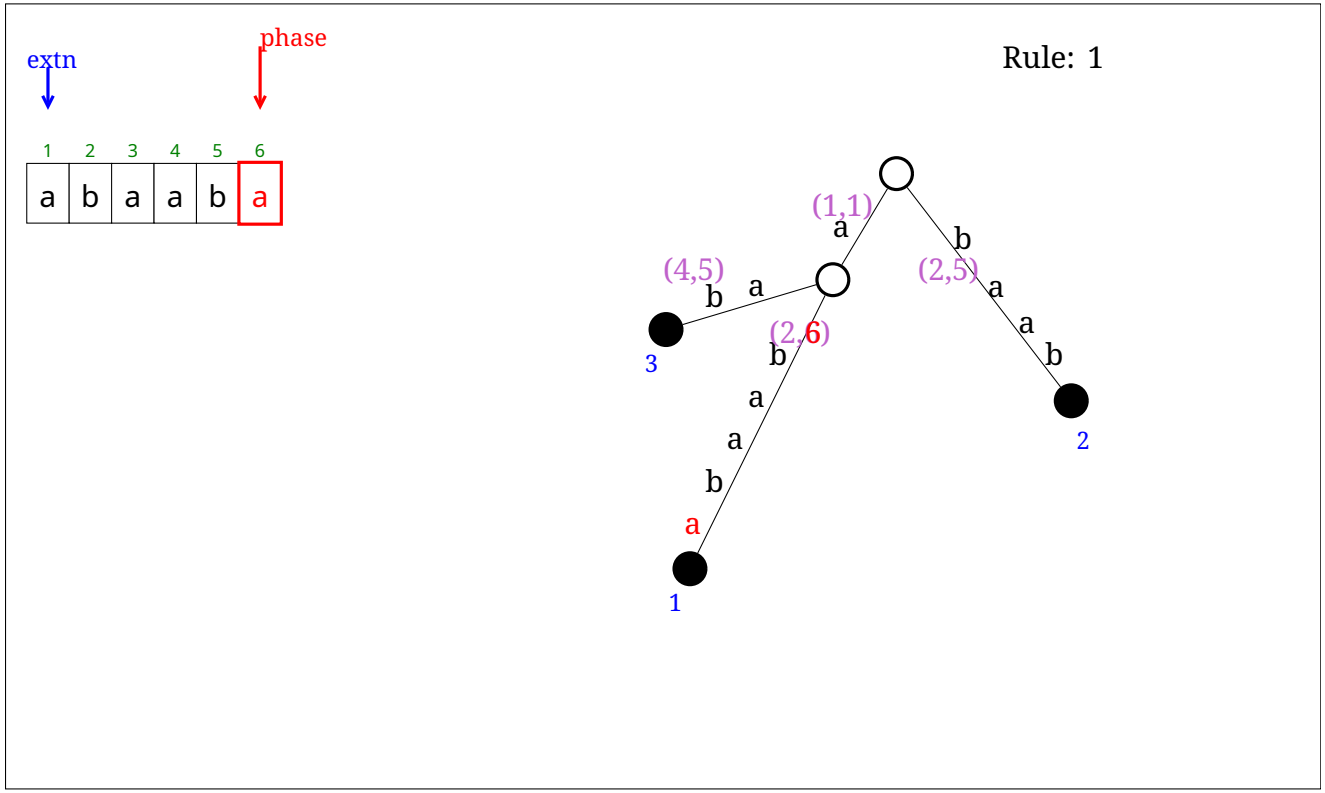
After Phase $i = 5$, Extension $j = 5$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

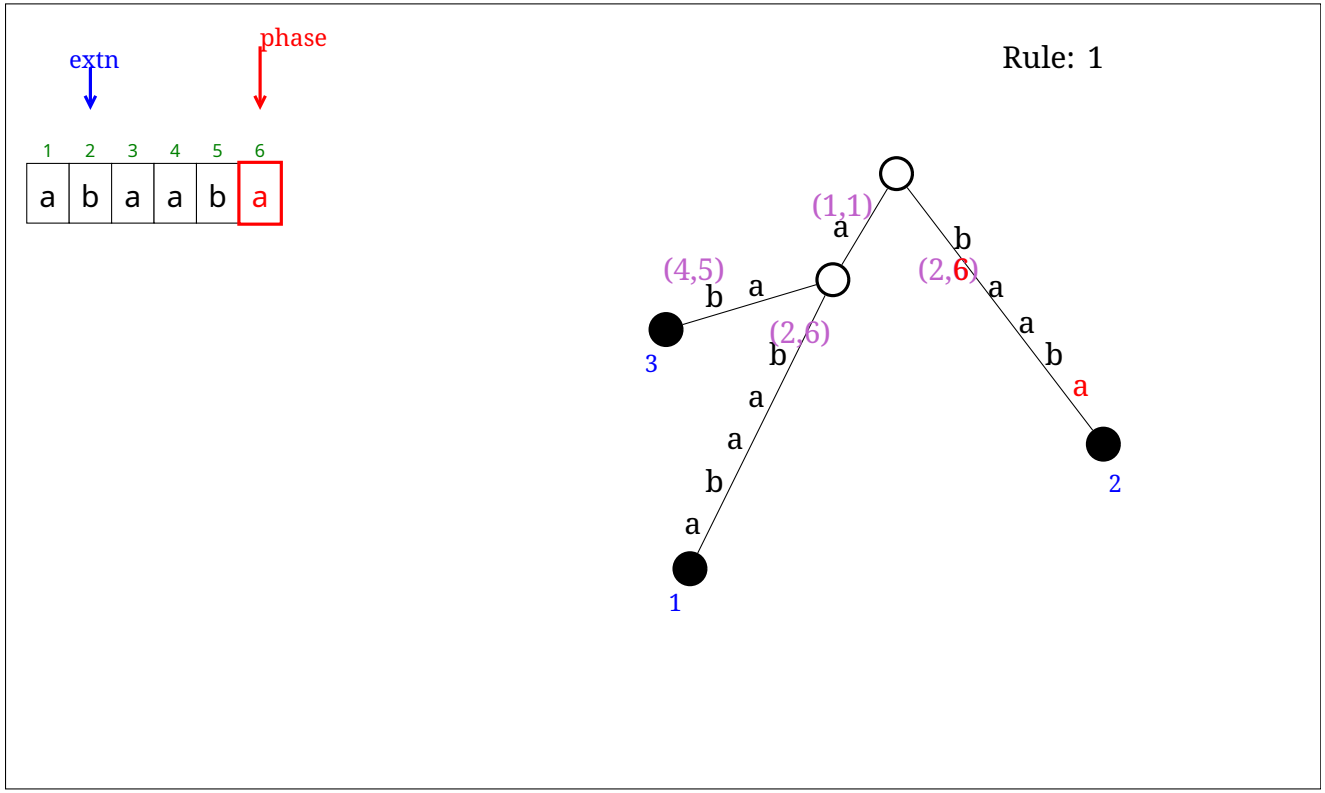
After Phase $i = 6$, Extension $j = 1$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

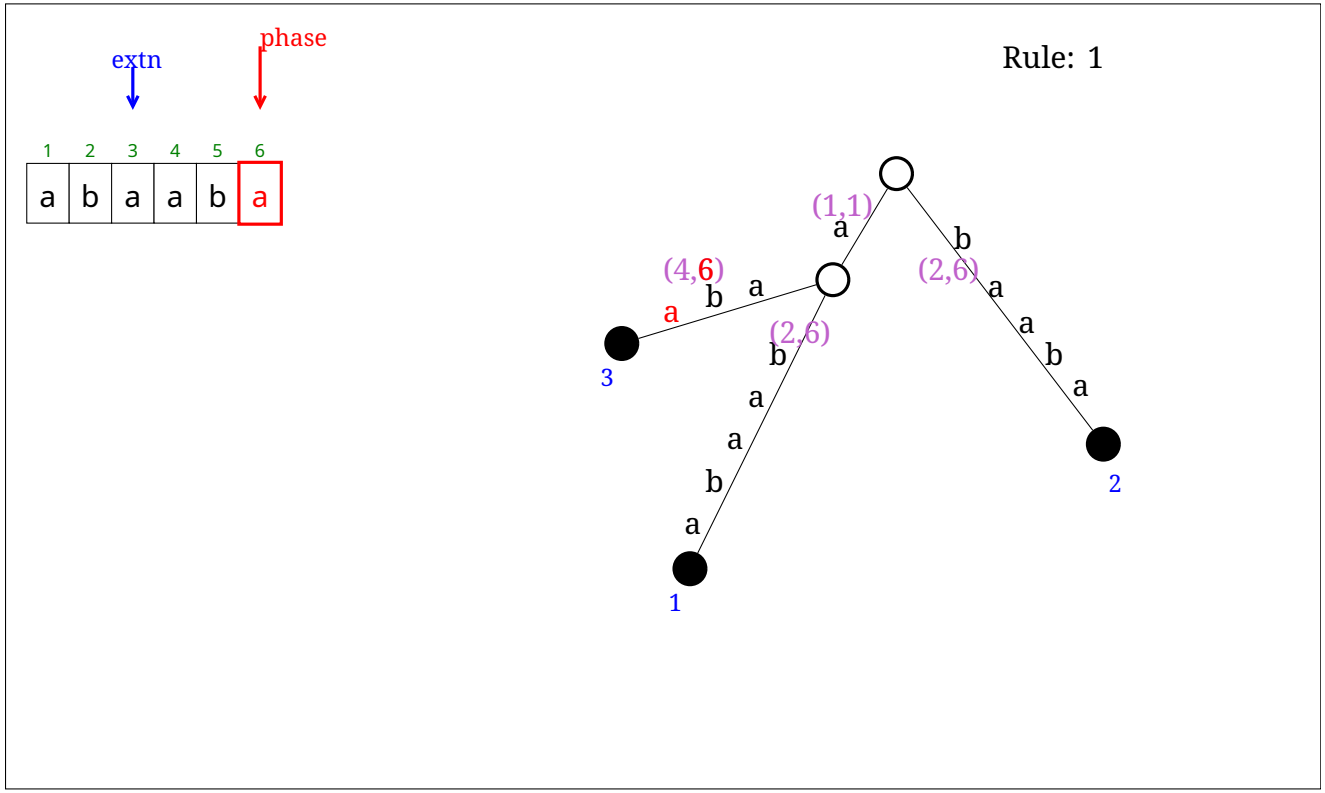
After Phase $i = 6$, Extension $j = 2$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

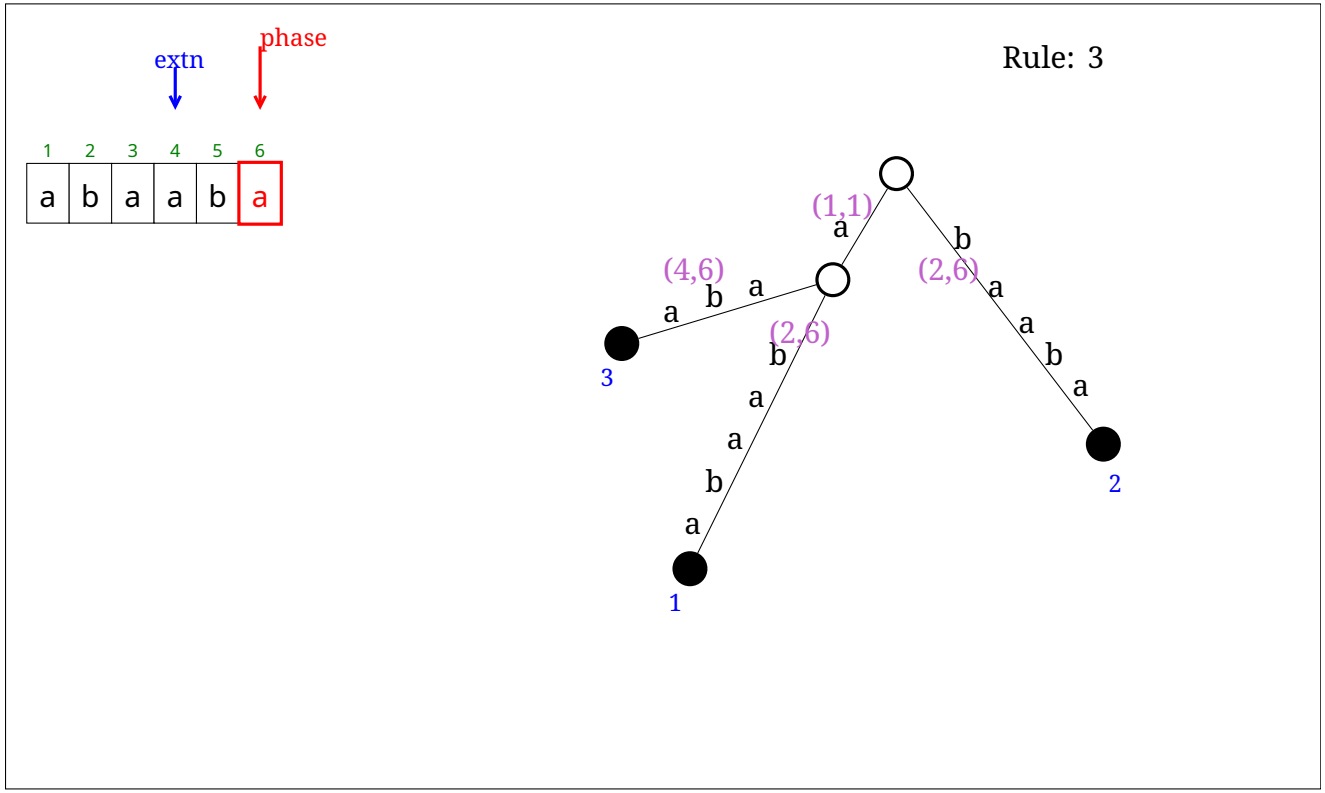
After Phase $i = 6$, Extension $j = 3$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

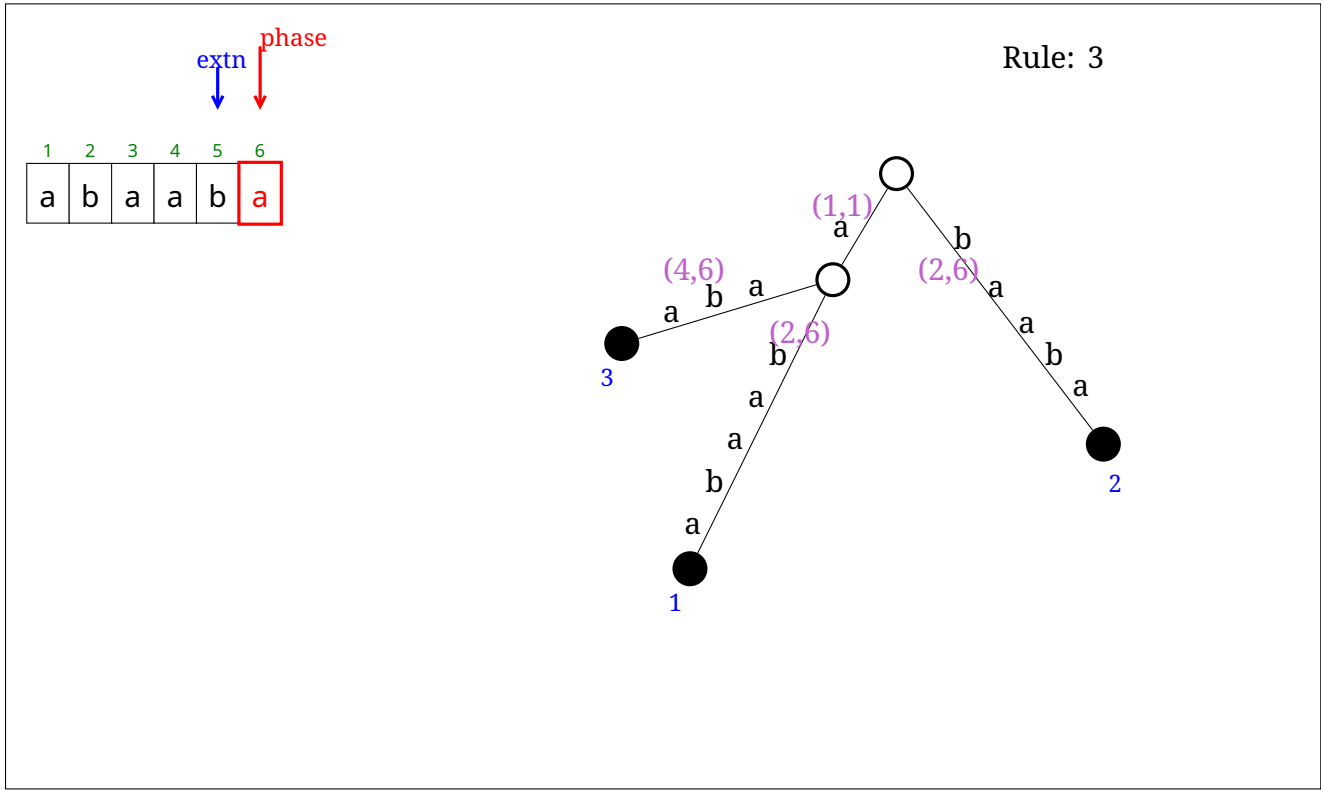
After Phase $i = 6$, Extension $j = 4$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

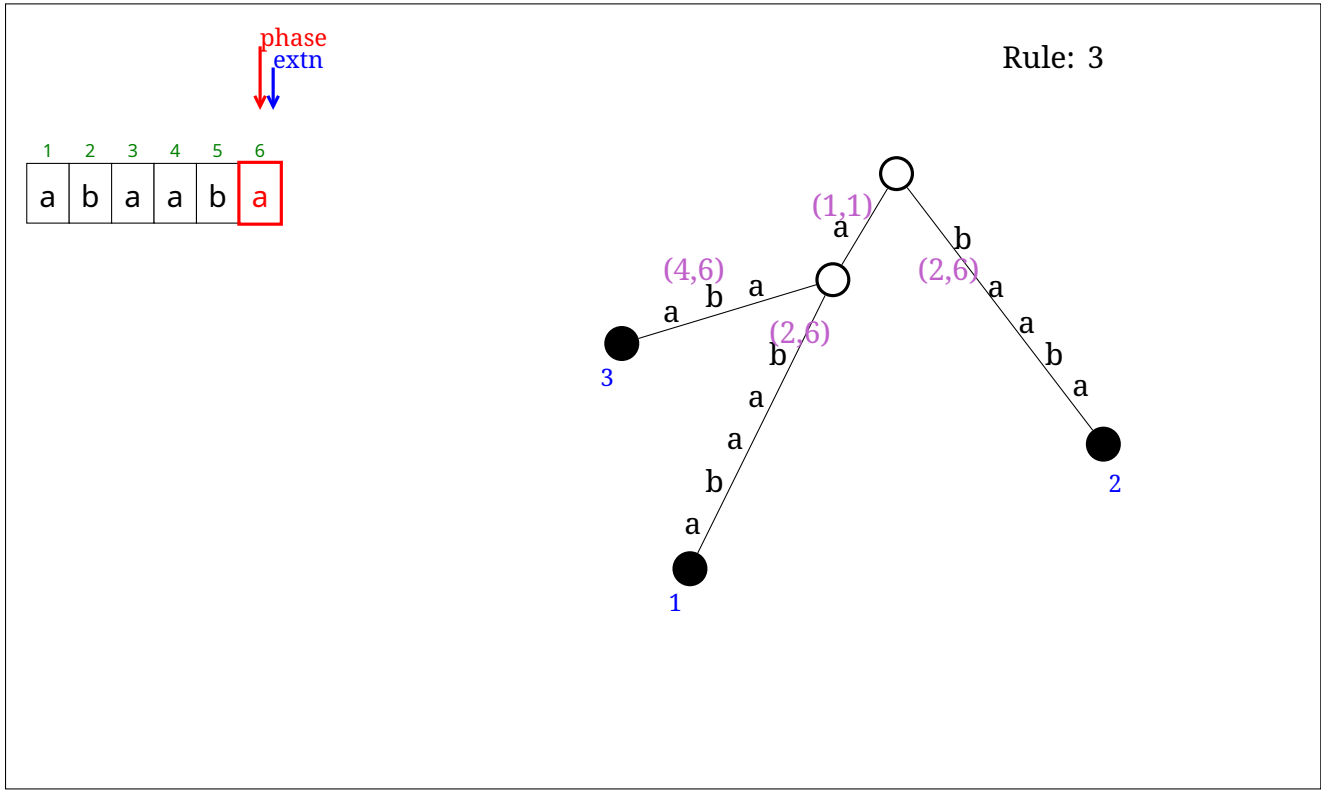
After Phase $i = 6$, Extension $j = 5$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

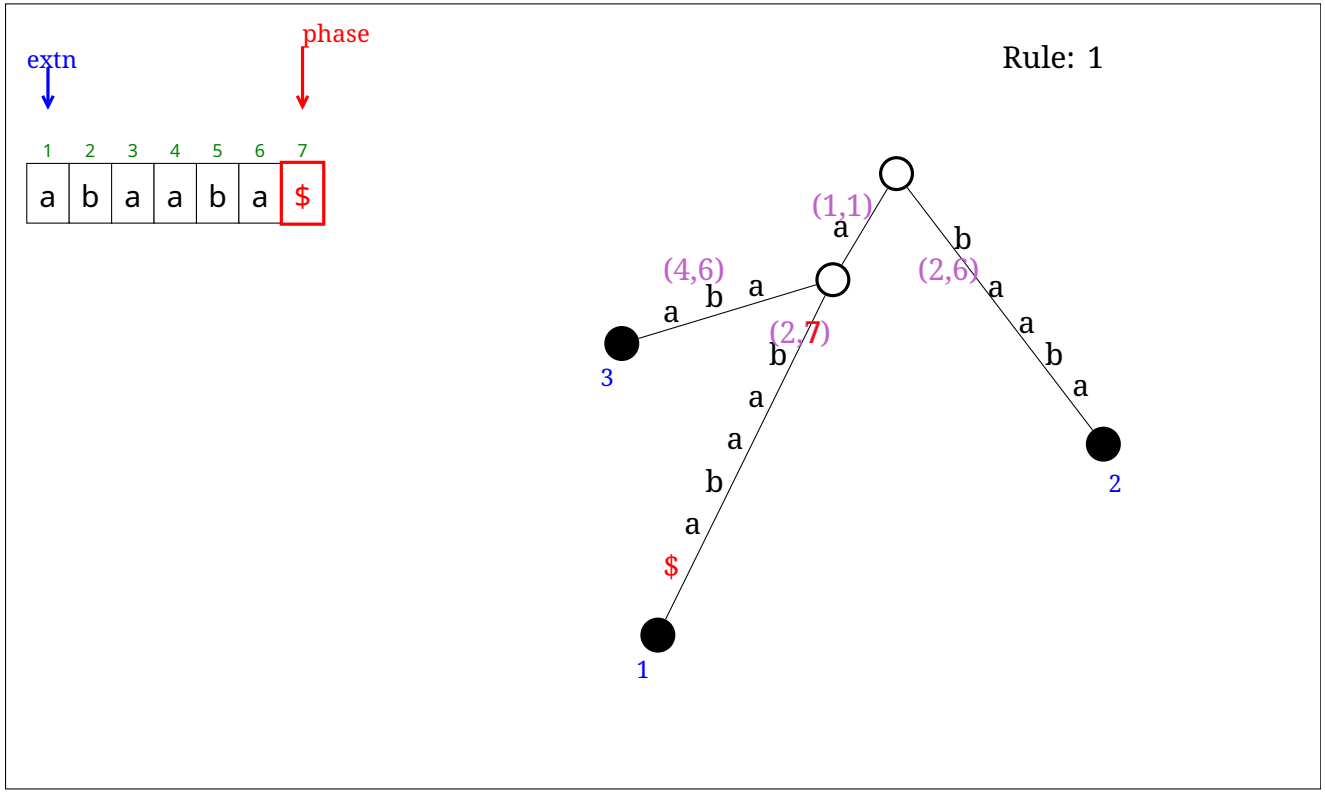
After Phase $i = 6$, Extension $j = 6$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

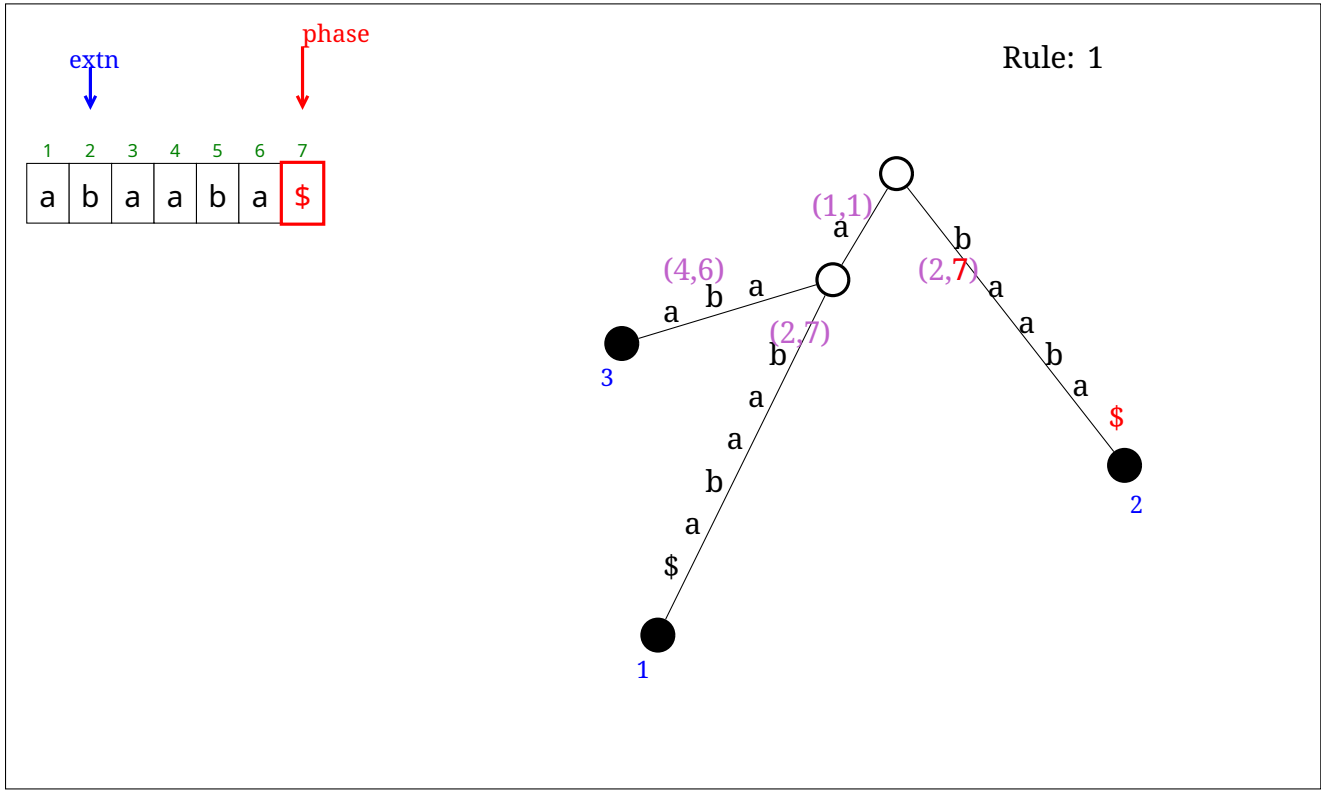
After Phase $i = 7$, Extension $j = 1$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

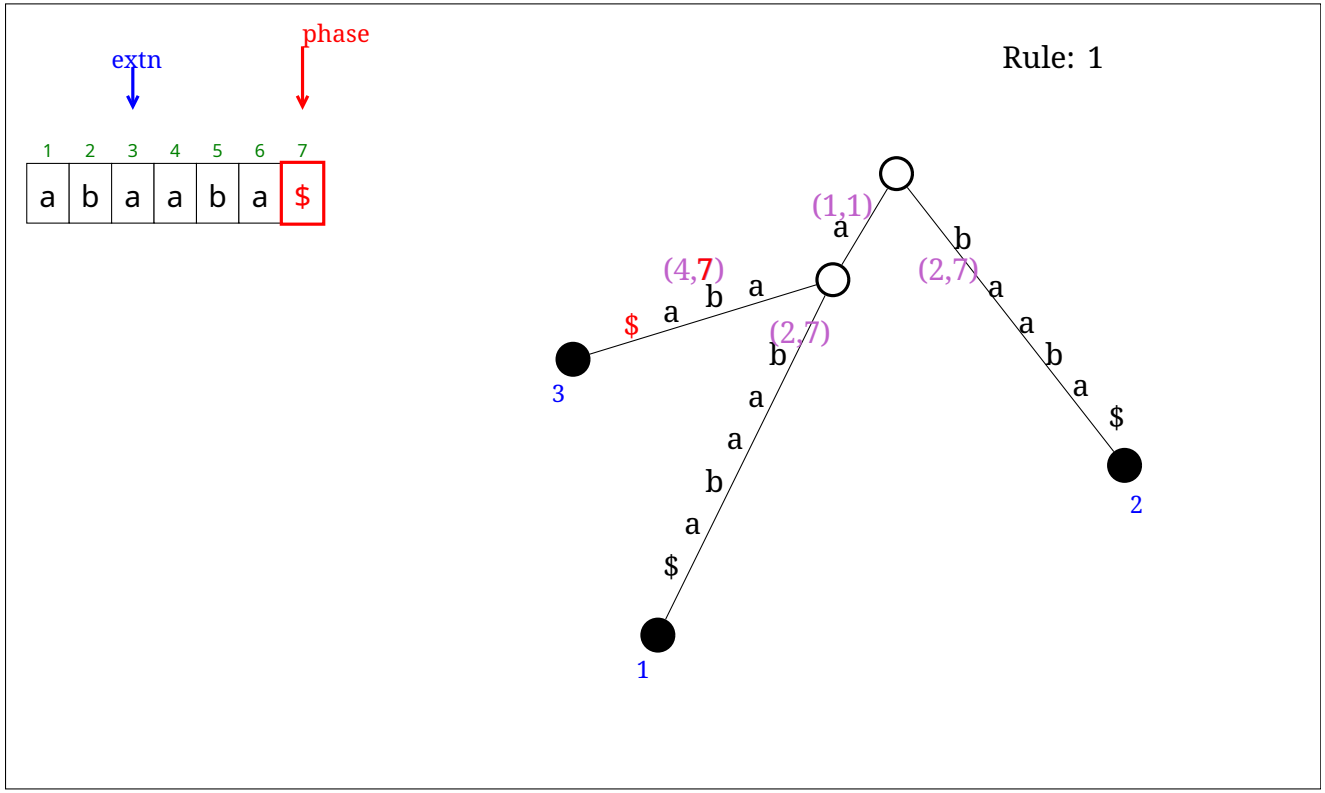
After Phase $i = 7$, Extension $j = 2$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

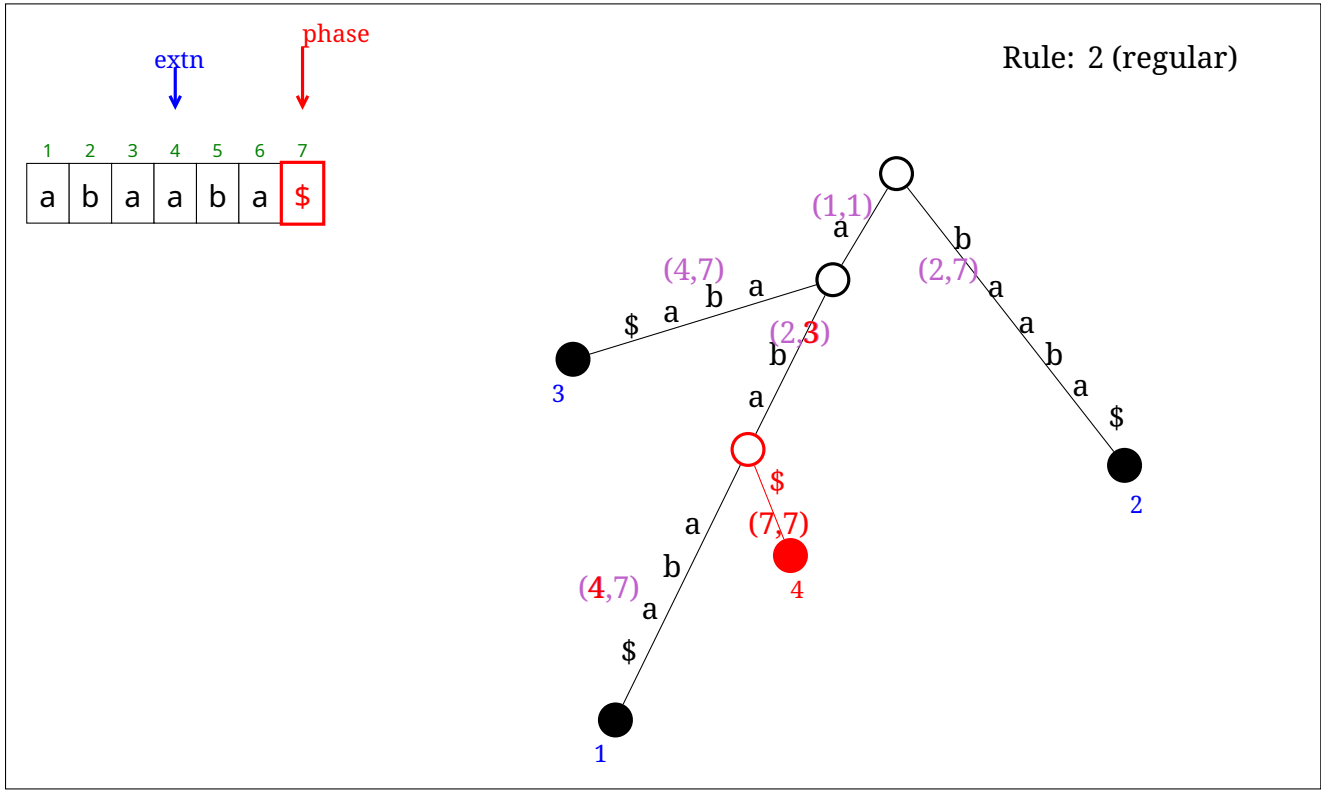
After Phase $i = 7$, Extension $j = 3$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

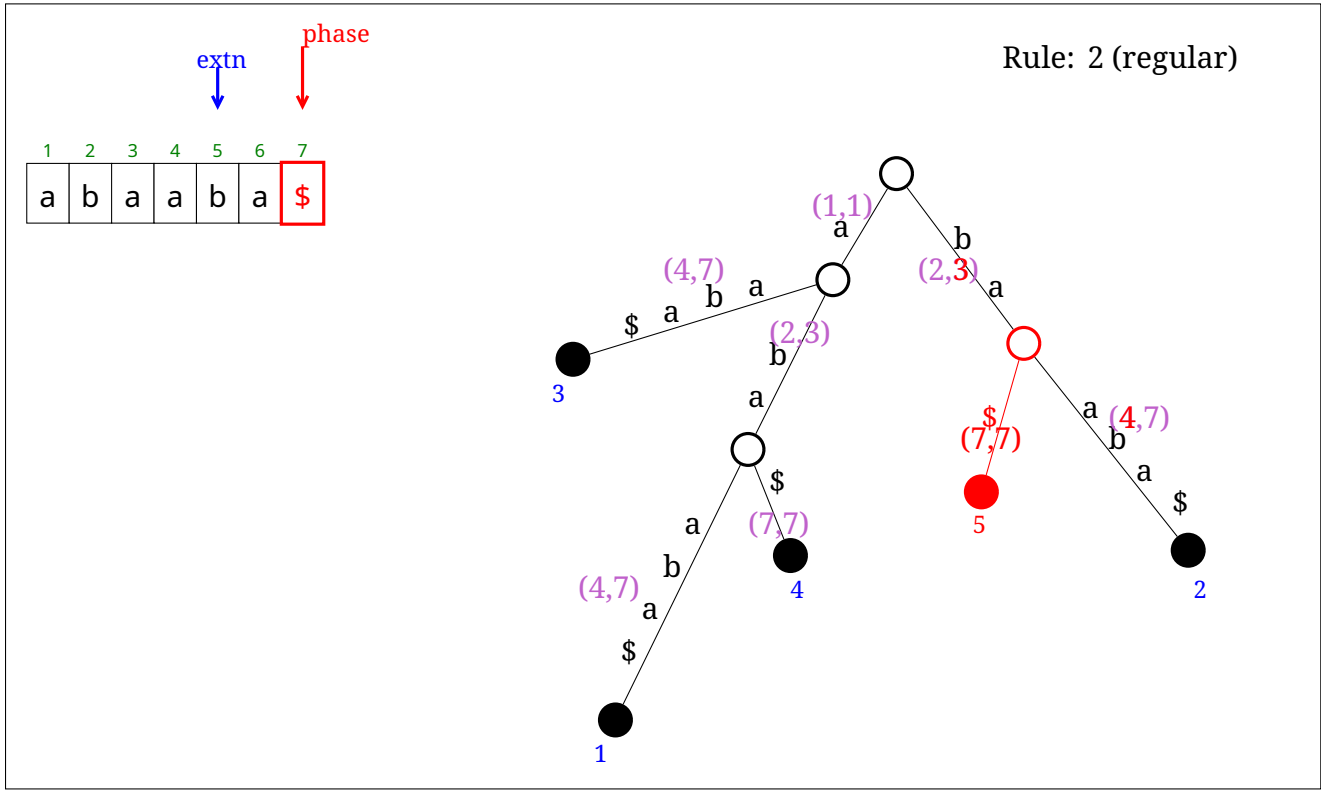
After Phase $i = 7$, Extension $j = 4$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

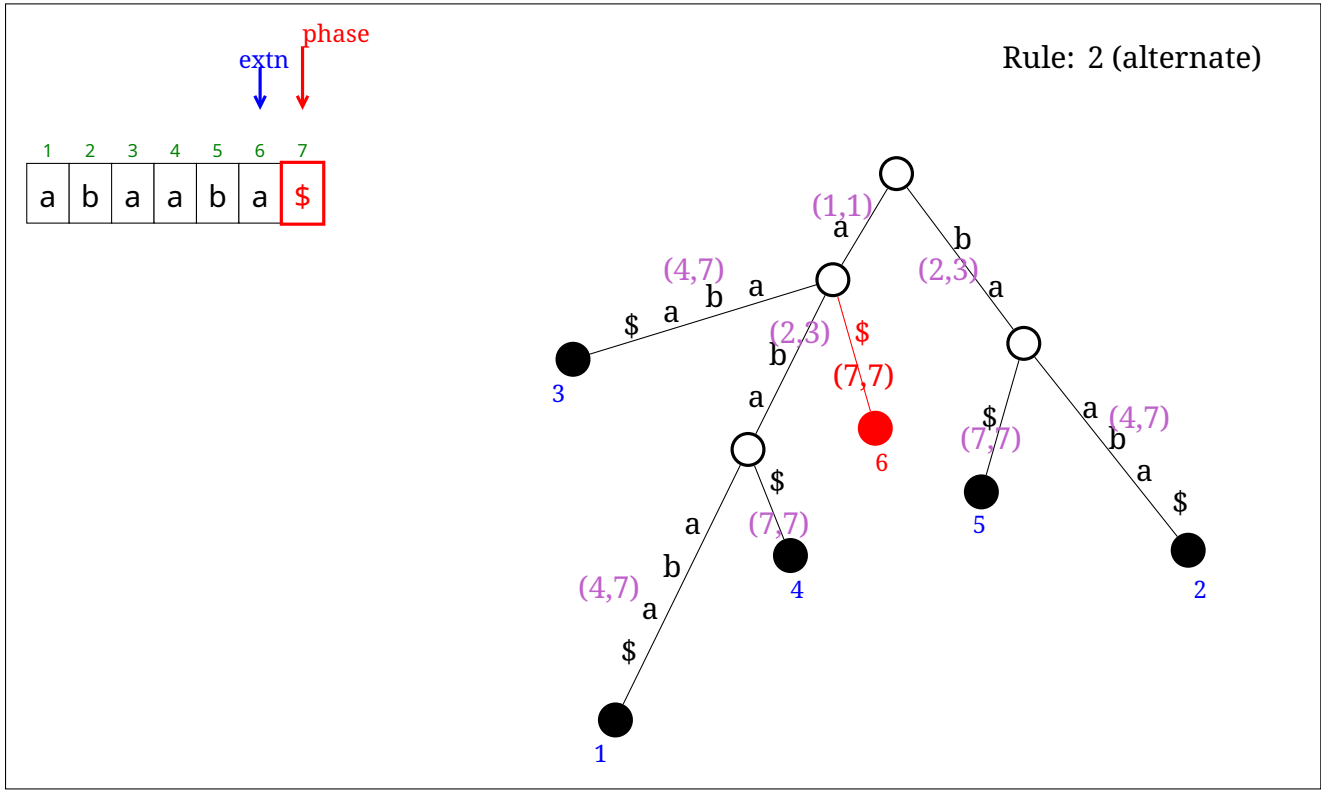
After Phase $i = 7$, Extension $j = 5$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

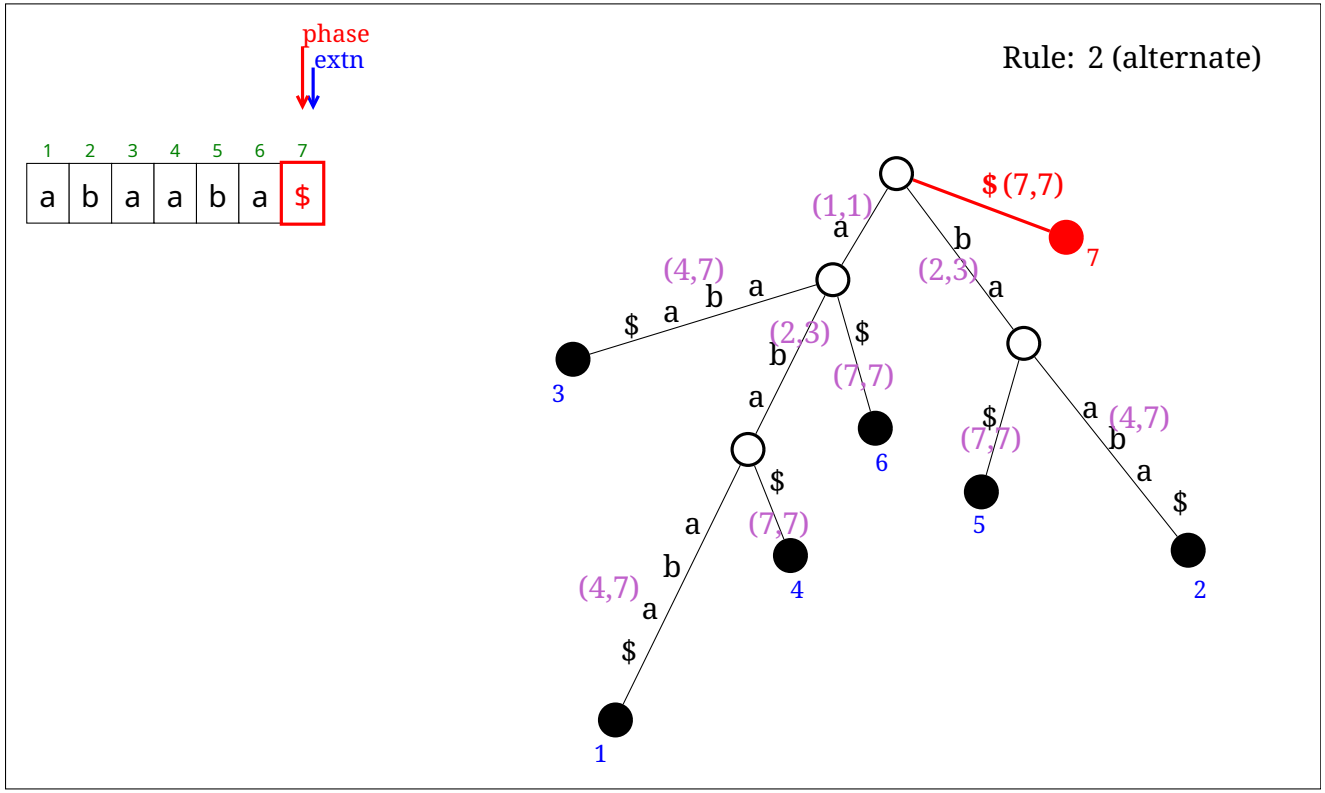
After Phase $i = 7$, Extension $j = 6$



Suffix extension rules – Example 2

$$\text{str}[1\ldots7] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{b}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\$}$$

After Phase $i = 7$, Extension $j = 7$



Speeding up tree traversal using suffix links

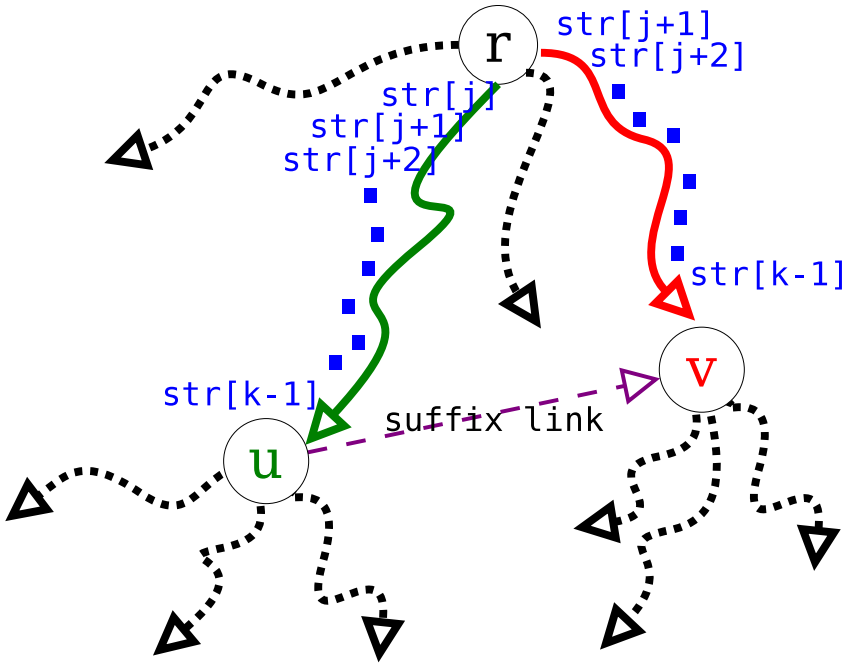
A suffix link is simply a **pointer** (i.e., a traversal-shortcut) between two internal nodes. Where feasible, they provide a speed up to perform suffix extensions.

Definition of a suffix link

- Let (u) and (v) be two internal nodes of an implicit suffix tree.
- Let the traversal from root node (r) to (u) define some substring **str**$[j..k-1]$.*
- Let the traversal from root node (r) to (v) define a substring **str**$[j+1..k-1]$.†
- Then the pointer from (u) to (v) defines a suffix link between those nodes.

*Note1: $1 \leq j \leq k-1$

†Note2: When $j = k-1$, the path from (r) to (u) yields a **single character** substring. This implies, the path from (r) to (v) will yield an empty substring. In other words, (v) and (r) are one and the same in this case.

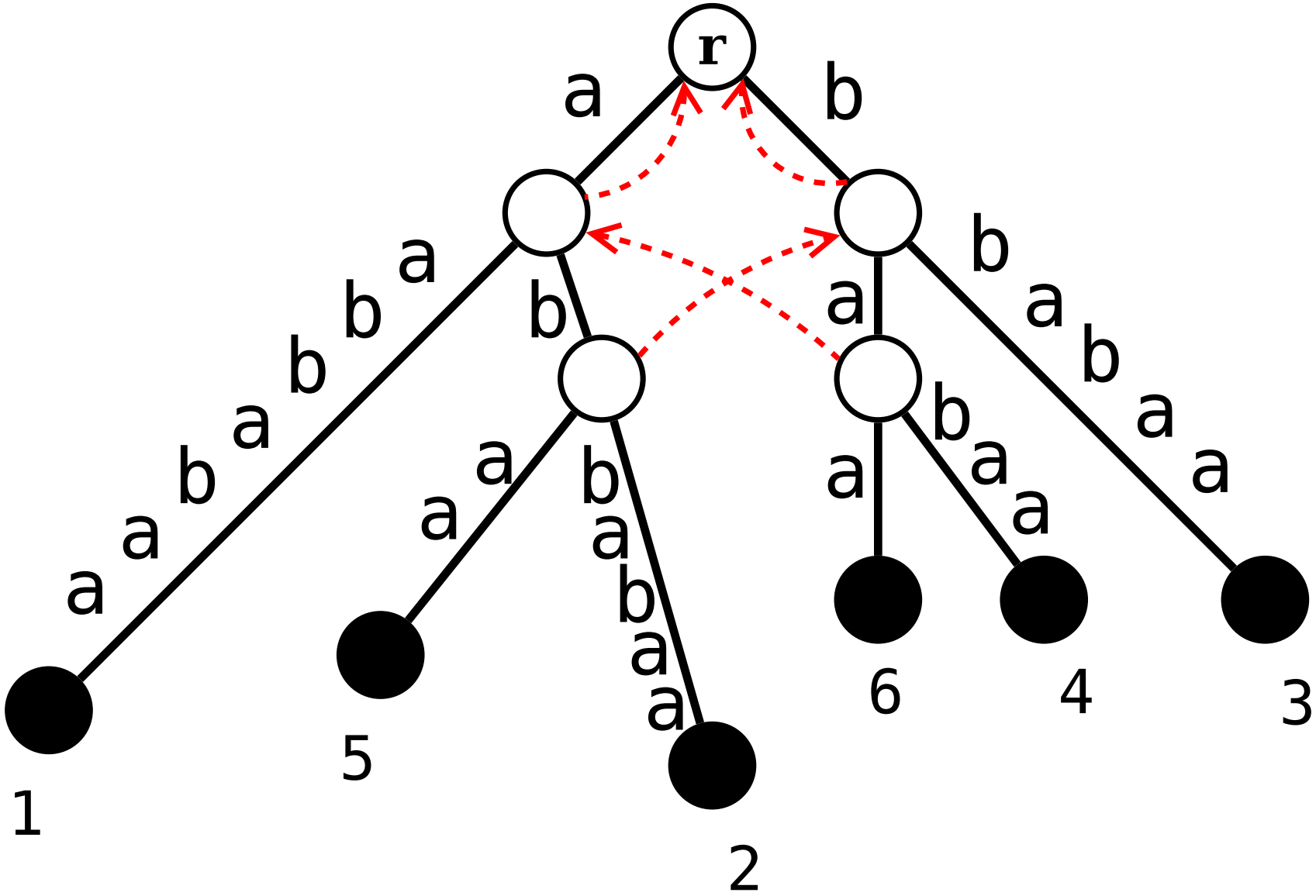


Key observation:

Every internal node of an implicit suffix tree has a suffix link out of it. (Root can be initialised as having a suffix link pointing to itself.)

Example – Implicit suffix tree with suffix links

1 2 3 4 5 6 7 8
str=a a b b a b a a



Following the trail of suffix links...

Terminology alert: "**Active node**" and "**Remainder**" (substring)

During any extension j of phase $i+1$ (meaning **str**$[i+1]$ is the new character being extended in this phase):

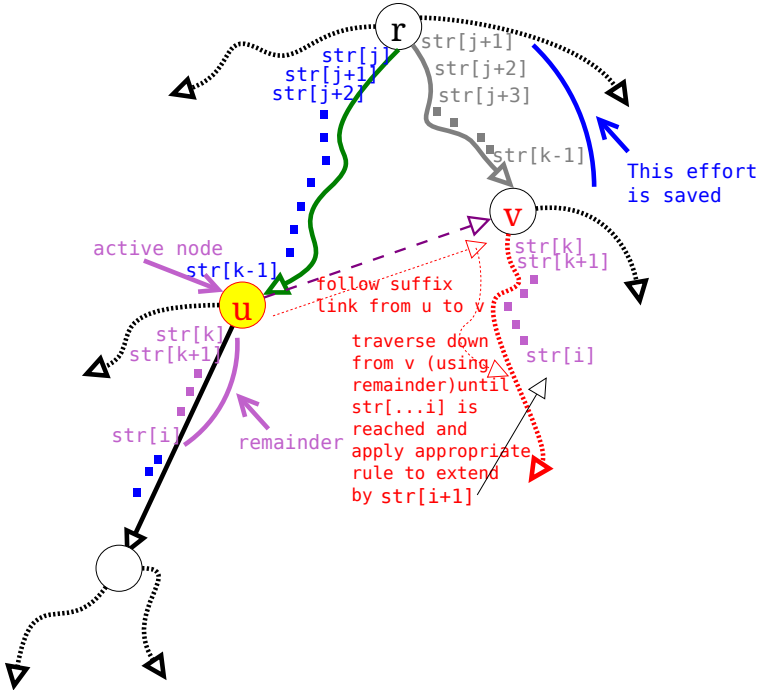
Active node: The pre-existing internal node (u), under whose direct edge the suffix extension rule is applied, is called the "**active node**".

Remainder (substring): If the path from the root (r) to the **active node** (u) spells out the substring **str**$[j \ldots k-1]$, where $j < k \leq i$, the substring **str**$[k \ldots i]$ on the edge below the active node (u) (and before the new character **str**$[i+1]$) is called the "**remainder**" substring.

At the start (Phase 1, Extension 1), the root (r) is the **active node** and the **remainder** is an empty (null) substring.

With these definitions, let us see an illustration of how one can save effort (meaning potentially make suffix extensions faster, than naively traversing from root each time) – see next slide.

...(cont'd) Following the trail of suffix links



Speeding up extension $j+1$ after completing extension j of phase $i+1$

- In each extension j of phase $i+1$, keep track of its **active node** (u) and **remainder** substring (k, i).
- Then, to carry out the next extension $j+1$ of phase $i+1$:
  - Traverse the suffix link from (u) to (v).
    - This saves the effort required to otherwise traverse naively from the root (r) to (v), giving us a free-ride for the str$[j+1 \ldots k-1]$.
  - From (v), traverse along the **remaining** substring str$[k \ldots i]$ to complete extension $j+1$ of phase $i+1$.

Note: When (v) = (r), this means you are doing a full extension (meaning, remainder is **str**$[j+1 \ldots i]$ and there is no saving achieved when this happens.)

Question: How/When does a new internal node during Ukkonen's algorithm get its suffix link?

Claim: When a new internal node (s) gets created in some extension j, its suffix link (s) to (some) (t) can ALWAYS be resolved in the very next $j+1$ extension of the same phase.

We will reason this from the point of view of Ukkonen's incremental algorithm

- If, in any suffix extension j of phase $i+1$, a new internal node (say) (s) is inserted below the active node (u), after the remainder substring (by applying rule 2 regular – Slide 19),
  - this implies, before (s) was inserted, the path $\texttt{str}[j..i]$ is continued by a character (say) x, where $x \neq \texttt{str}[i+1]$.
- then, in the very next suffix extension $j+1$ of the same phase $i+1$:
  1. **either** the path $\texttt{str}[j+1..i]$ continues **ONLY VIA** character x.
     - ★ which implies, rule 2 regular is applicable again: i.e., a new internal node (say) (t) must also be created, after $\texttt{str}[j+1..i]$, that branches to the new leaf $j+1$ via character $\texttt{str}[i+1]$.
  2. **or** the path $\texttt{str}[j+1..i]$ already ends in an existing internal node (t),
     - ★ ... with one branch below extending via character x
     - ★ ... and one (or more) branch(es), via other character(s).

Using four key tricks during suffix tree construction, Ukkonen's algorithm yields a linear-time constuction.

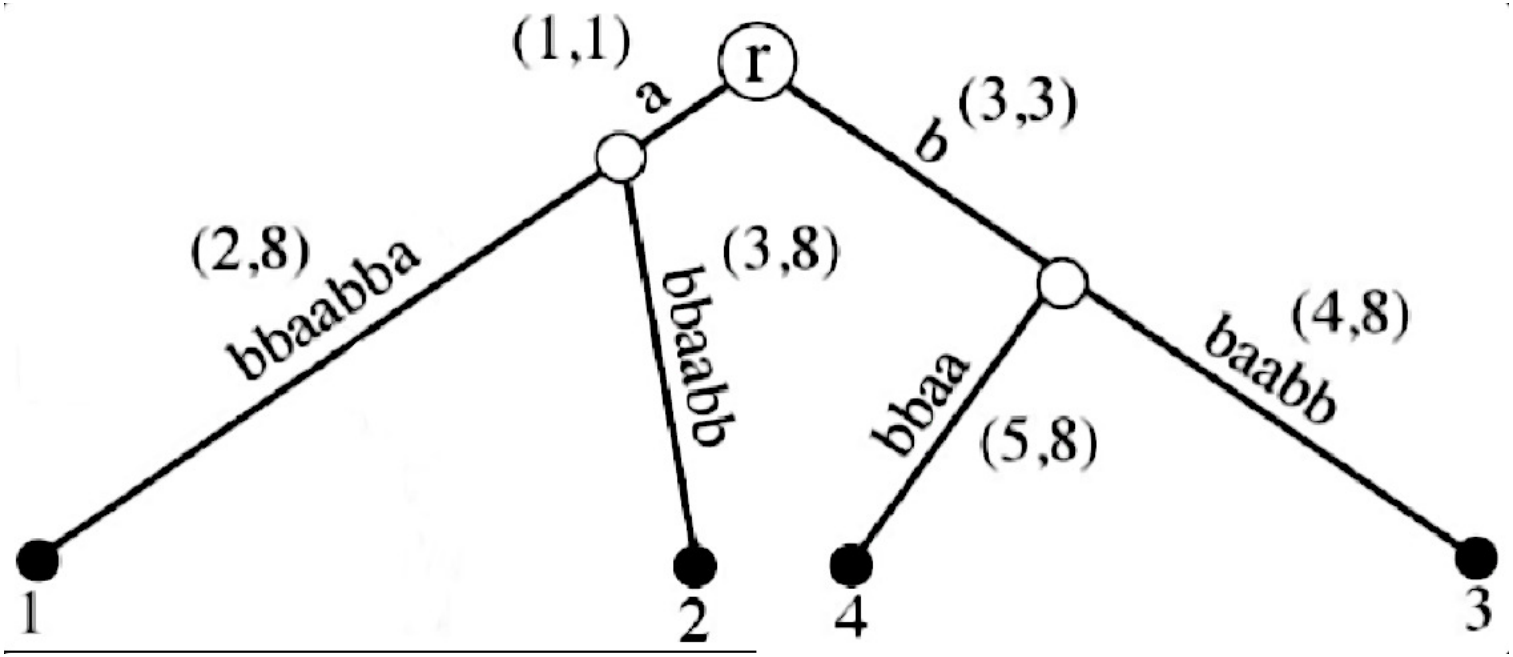
Now all that remains is to apply four **`implementation tricks`** to build on this general incremental procedure, to get a guaranteed linear time algorithm.

We will study them one by one in the slides below.

Implementation Trick 1: Space-efficient representation of edge-labels/substrings

Recall, given any string **str**$[1..n]$, any substring can be represented by just two numbers: (**start-index**,**end-index**). Thus, the entire Ukkonen algorithm is processed using this space-efficient ($O(n)$-space) representation.

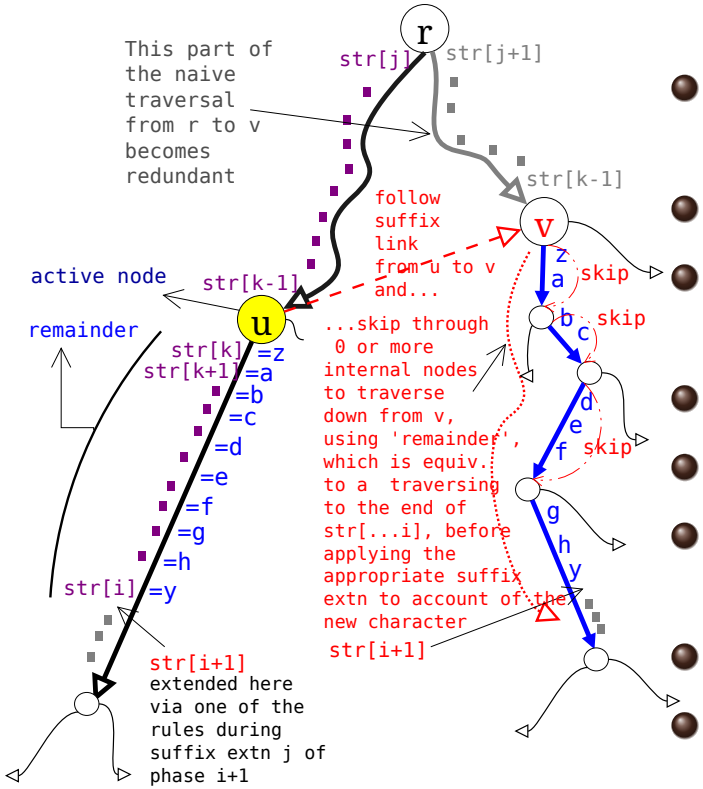
Below is an implicit suffix tree of the string **str**=a a b b a a b b (1 2 3 4 5 6 7 8), using (**start-index**,**end-index**) edge label representation.



Note: Even **remainder** of any extn/phase is a substring, and hence is kept track using start and end indexes wrt the string.

Implementation Trick 2: Skip-Count trick

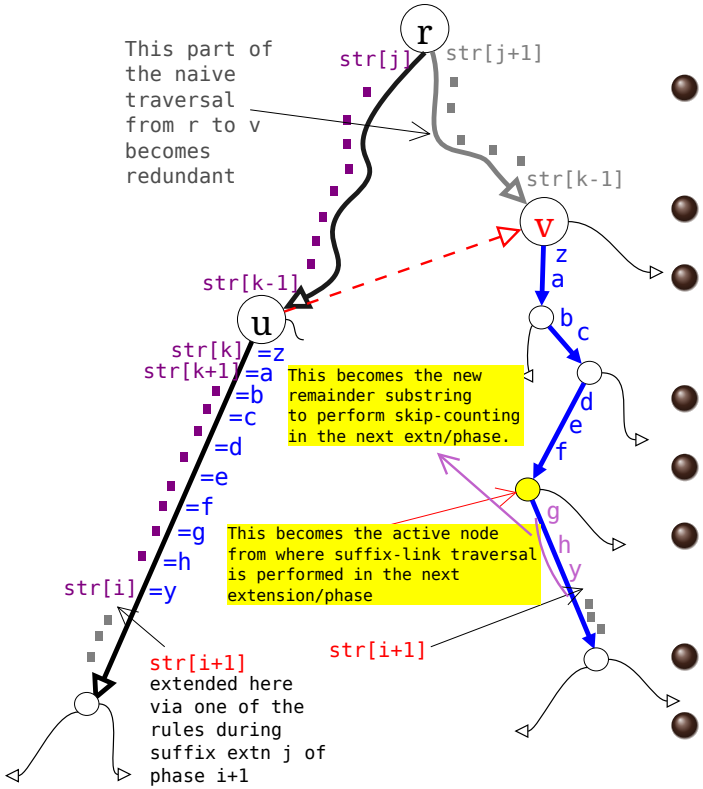
Using a space-saving representation of edges seen in the previous slide, traversals in an implicit suffix tree can skip multiple characters at once instead of comparing one character at a time. This allows fast traversals during extensions, while keeping track of the total length of the skipped substring. For example:



- Let the **remainder** substr be $\mathtt{str}[k..i]$=z a b c d e f g h y (1 2 3 4 5 6 7 8 9 10)
- From node (v), ask how many characters representing the edge starting with z... Here. it is 2.
- Since $10 > 2$, end not reached, so skip to the node receiving that edge.
- In $\mathtt{str}[k..i]$, the 3rd $(3 = 2 + 1)$ character is b.
- Again, ask how many characters representing the edge starting with b.... Again it is 2.
- Since $10 > 2 + 2$, skip to the node receiving that edge.
- From $\mathtt{str}[k..i]$, the 5th $(5 = 2 + 2 + 1)$ character is d.
- Again, ask how many characters representing the edge starting with d.... It is 3.
- Since $10 > 2 + 2 + 3$, skip to the node receiving that edge.
- ...and so on until the node beyond which further skips are not possible/necessary.

Implementation Trick 2: Skip-Count trick

Using a space-saving representation of edges seen in the previous slide, traversals in an implicit suffix tree can skip multiple characters at once instead of comparing one character at a time. This allows fast traversals during extensions, while keeping track of the total length of the skipped substring. For example:



- Let the **remainder** substr be $\texttt{str}[k..i]$=

  1 2 3 4 5 6 7 8 9 10
  z a b c d e f g h y
- From node (v), ask how many characters representing the edge starting with z... Here. it is 2.
- Since $10 > 2$, end not reached, so skip to the node receiving that edge.
- In $\texttt{str}[k..i]$, the 3rd $(3 = 2 + 1)$ character is b.
- Again, ask how many characters representing the edge starting with b.... Again it is 2.
- Since $10 > 2 + 2$, skip to the node receiving that edge.
- From $\texttt{str}[k..i]$, the 5th $(5 = 2 + 2 + 1)$ character is d.
- Again, ask how many characters representing the edge starting with d.... It is 3.
- Since $10 > 2 + 2 + 3$, skip to the node receiving that edge.
- ...and so on until the node beyond which further skips are not possible/necessary.

Once skip-counting completed, update **active node** and **remainder** (in line with their definitions on slide 24).

Implementation Trick 3: Early termination of a phase when Rule 3 is first observed (aka **'Phase Showstopper' rule!)**

In any phase $i+1$, if **rule 3** extension (refer slide 20) applies in some suffix extension j, then extensions $j+1, j+2, \ldots i+1$ will all use **rule 3**. Because:

- when **rule 3** is used in extension j, implies the path corresponding to **str**$[j..i]$ continues with the character **str**$[i+1]$.
- This implies, the path corresponding to the substring **str**$[j+1..i]$ also continues with the character **str**$[i+1]$
- Similarly, this remains true for all subsequent extensions.

Punchline: Since **rule 3** requires **no further action**, extensions in this phase can **STOP** early upon first encountering **rule** 3, and the algorithm can directly start the extensions for the next phase.

Implementation Trick 4: rapid leaf extension trick (slide 1 of 3)

Observation– In Ukkonen's algorithm, once a leaf, always a leaf

If at some phase i in Ukkonen's algorithm, a leaf is created and labeled j (denoting a suffix $\mathtt{str}[j..i]$ of the prefix $\mathtt{str}[1..i]$), then that leaf will remain a leaf in all subsequent phases ($> i$).

Why?

- Leaf node, when created (via **rule 2**) always stores as its label, the starting index j denoting where the corresponding suffix starts.
- In subsequent phases, whenever this suffix is extended at the leaf (via **rule 1**), only the edge-label connecting the leaf gets updated, and not the leaf node label.

Implementation Trick 4: rapid leaf extension trick (slide 2 of 3)

- Define the variable $\texttt{last}_{j_i}$ that stores the index of the **last** suffix extension j for phase i that was performed either using Rule 1 or Rule 2 (be it regular or alternate).
  - In other words, in phase i, all suffix extensions $j = \{1, 2, \ldots, \texttt{last}_{j_i}\}$ were either Rule 1 or Rule 2 extensions.

Claim: In the next phase $i + 1$, all these suffix extensions $j = \{1, 2, \ldots, \texttt{last}_{j_i}\}$ will be Rule 1 extensions. (Why? Listen to the lecture.)

- It follows from the above that $\texttt{last}_{j_i} \leq \texttt{last}_{j_{i+1}}$.
- Note that Rule 1 only requires updating the end point of the edge label to a leaf from i to $i + 1$ (which is always the same as the phase number – see Slide 18).
- Thus, on each edges leading into a leaf, a simple $global_end$ variable that is in sync with the phase number can be used and avoid performing **explicitly** carrying out suffix extensions $j = \{1, 2, \ldots, \texttt{last}_{j_i}\}$, and instead do them **implicitly** (and all simultaneously) in $\mathcal{O}(1)$-time, discussed in next slide.

Implementation Trick 4: rapid leaf extension trick (slide 3 of 3)

- From the space-efficient representation (see slide 28) each edge label contains two integers: (`start-index`, `end-index`).

Implementation Trick 4: rapid leaf extension trick (slide 3 of 3)

- From the space-efficient representation (see slide 28) each edge label contains two integers: (`start-index`, `end-index`).
- In any extension j of any phase i, a leaf node is created (when either Rule 2 regular or alternate is applied).

Implementation Trick 4: rapid leaf extension trick (slide 3 of 3)

- From the space-efficient representation (see slide 28) each edge label contains two integers: (`start-index`, `end-index`).
- In any extension j of any phase i, a leaf node is created (when either Rule 2 regular or alternate is applied).
- The edge connecting the leaf node when first created starts off as (`start-index`, `end-index`)$= (i, i)$.

Implementation Trick 4: rapid leaf extension trick (slide 3 of 3)

- From the space-efficient representation (see slide 28) each edge label contains two integers: (`start-index`, `end-index`).
- In any extension j of any phase i, a leaf node is created (when either Rule 2 regular or alternate is applied).
- The edge connecting the leaf node when first created starts off as (`start-index`, `end-index`)$= (i, i)$.
- Instead, when any leaf is created for the first time, avoid EXPLICITLY assigning `end-index`$= i$ (which will require an explicit leaf extension via Rule 1 subsequently). Instead, 'cleverly' assign the edge label IMPLICITLY as: (`start-index`, `end-index`)$= (i, global_end)$, where this $global_end$ always is kept same as phase number.

Implementation Trick 4: rapid leaf extension trick (slide 3 of 3)

- From the space-efficient representation (see slide 28) each edge label contains two integers: (`start-index`, `end-index`).
- In any extension j of any phase i, a leaf node is created (when either Rule 2 regular or alternate is applied).
- The edge connecting the leaf node when first created starts off as (`start-index`, `end-index`)$= (i, i)$.
- Instead, when any leaf is created for the first time, avoid EXPLICITLY assigning `end-index`$= i$ (which will require an explicit leaf extension via Rule 1 subsequently). Instead, 'cleverly' assign the edge label IMPLICITLY as: (`start-index`, `end-index`)$= (i, global_end)$, where this $global_end$ always is kept same as phase number.
- When phase changes from i to $i + 1$, updating $global_end$ implies there is....

Implementation Trick 4: rapid leaf extension trick (slide 3 of 3)

- From the space-efficient representation (see slide 28) each edge label contains two integers: (`start-index`, `end-index`).
- In any extension j of any phase i, a leaf node is created (when either Rule 2 regular or alternate is applied).
- The edge connecting the leaf node when first created starts off as (`start-index`, `end-index`)$= (i, i)$.
- Instead, when any leaf is created for the first time, avoid EXPLICITLY assigning `end-index`$= i$ (which will require an explicit leaf extension via Rule 1 subsequently). Instead, 'cleverly' assign the edge label IMPLICITLY as: (`start-index`, `end-index`)$= (i, global_end)$, where this $global_end$ always is kept same as phase number.
- When phase changes from i to $i + 1$, updating $global_end$ implies there is....
- ...**no additional explicit work** to implement extensions $j = 1$, $j = 2$, $\ldots j =$ **last**$_{j_i}$. They are all get updated simultaneously when $global_end$ gets updates (with $\mathcal{O}(1)$-effort).

Implementation Trick 4: rapid leaf extension trick (slide 3 of 3)

- From the space-efficient representation (see slide 28) each edge label contains two integers: (`start-index`, `end-index`).
- In any extension j of any phase i, a leaf node is created (when either Rule 2 regular or alternate is applied).
- The edge connecting the leaf node when first created starts off as (`start-index`, `end-index`)$= (i, i)$.
- Instead, when any leaf is created for the first time, avoid EXPLICITLY assigning `end-index`$= i$ (which will require an explicit leaf extension via Rule 1 subsequently). Instead, 'cleverly' assign the edge label IMPLICITLY as: (`start-index`, `end-index`)$= (i, global_end)$, where this $global_end$ always is kept same as phase number.
- When phase changes from i to $i + 1$, updating $global_end$ implies there is....
- ...**no additional explicit work** to implement extensions $j = 1$, $j = 2$, $\ldots j = \mathtt{last}_{j_i}$. They are all get updated simultaneously when $global_end$ gets updates (with $\mathcal{O}(1)$-effort).
- Hence, the **`EXPLICIT suffix extensions`** for any next phase i+1 start from the position $j = \mathtt{last}_{j_i} + 1$ and end when either a Rule 3 extension is observed or when the phase ends naturally.

Putting trick 4 pieces together – procedure to handle extensions in any single phase

Summarizing trick 4 , for any phase $i+1$, the extension procedure is as follows:

1. Increment $global_end$ index to i+1.
   - With just this operation, suffix extensions 1 to $\mathtt{last}_{j_i}$ of phase $i+1$ are instantaneously completed without any additional work.
2. Explicitly compute successive extensions starting from $j = \mathtt{last}_{j_i} + 1$, until some position $q \leq i+1$ where the phase prematurely terminates (after first encountering Rule 3 extension), or all extensions are completed without encountering Rule 3 (where q is set to $i+2$).
3. To prepare for next phase $i+2$, set $\mathtt{last}_{j_{i+1}}$ to $q-1$ and repeat the procedure above, until all phases are complete.

Key Observation

Any two consecutive phases $(i \rightarrow i+1)_{\forall 0 \leq i < n}$ share **at most** ($\leq$) one position q where an EXPLICIT extension is carried out.

Creating the final suffix tree from $\texttt{implicitST}_n$ for $\texttt{str}[1...n]$

The final suffix tree from its implicit version can be computed in $O(n)$-time as follows:

- First add a string terminal symbol $\$$ to the end of **str**, i.e. $\texttt{str}[1...n]\$$.
- Continue one more phase on $\texttt{implicitST}_n$ to account for this new character.
- The effect is that no suffix is now a prefix in $\texttt{implicitST}_n$.
- So each suffix of $\texttt{str}[1..n]$ gets appended by $\$$, yielding the **regular/explicit** suffix tree.

Ukkonen's algorithm runs in $O(n)$ time for a string **str**[1..n]

- There are only n phases in the algorithm.
- Each phase shares at most 1 explicit suffix extension (see slide 34).
- Hence, total number of explicit suffix extensions is at most $2n$.
- To quantify the effort in each extension:
  - If the node in the tree after which the suffix extension **str**$[j-1..i]$ ends is at depth d from (r)
  - Then (u) is at depth at least $d-1$.
  - This implies the node receiving its suffix link, (v) is at least $d-2$
- Total number of skips over **all phases** is $O(n)$
- From this, it follows, Ukkonen takes $O(n)$-time

Alternate reasoning will be discussed during the lecture

An equivalent reasoning can be explored using the indexes of characters (instead of depth) leading into active nodes between successive iterations. Specifically, these indexes (unlike depths) between iterations have to be monotonically increasing. This bounds the total number of hops during skip-counts over all explicit suffix extensions by n.

Next...

Lossless Data Compression

-=o0o=-

END

-=o0o=-